

图像增强1-20250306

题量: 2 满分: 100

智能分析

100分

作答时间: 2025-03-07 11:55 至 2025-03-14 11:55

一. 简答题（共2题，100分）

1. (简答题)

(图像增强)-请用幂指数变换对下图进行增强。
(内容请包含 1. 算法原理; 2. 关键代码; 3. 结果图; 4. 结果分析)



1.zip
292.12 KB

我的答案:

50 分

1、算法原理

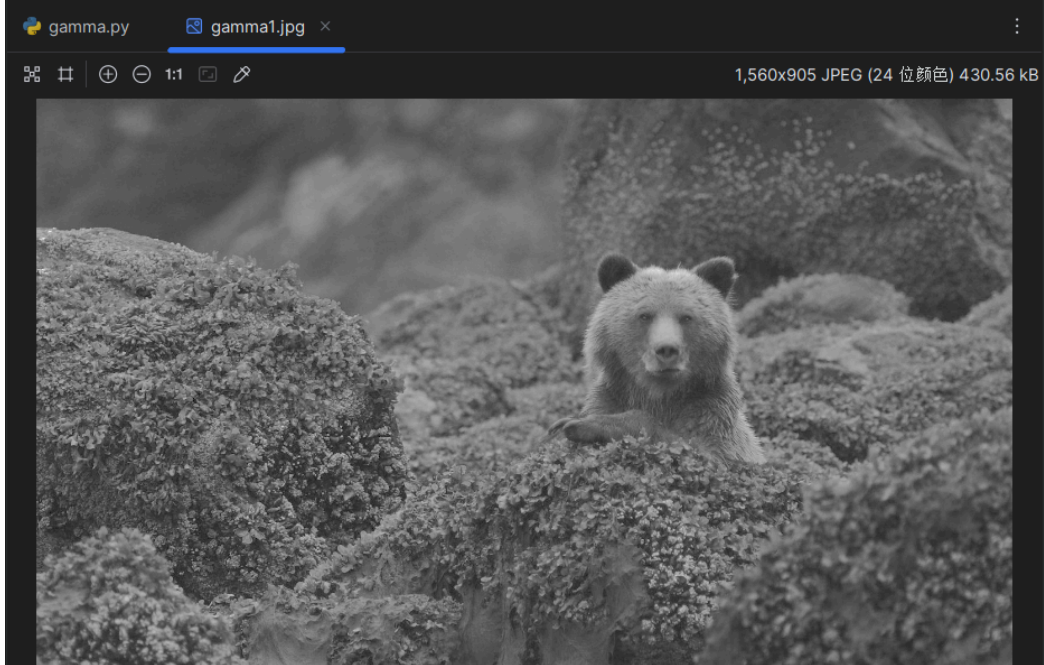
使用numpy模块创建新图像，各点像素值（float32类型）初值均取0（纯黑色）。
对图像的每个像素值，应用Gamma校正公式，计算新的像素值：
$$s = c \cdot r^\gamma$$

其中，取 $\gamma < 1$ ，图像整体变亮，暗部细节更清晰。
再使用cv2模块normalize函数的NORM_MINMAX方法，将图像的像素值线性地映射到0~255，而不改变图像的相对对比度。

2、关键代码

```
def gamma(c,gamma,image):  
    h,w = image.shape[0],image.shape[1]  
    t_img = np.zeros((h,w),dtype=np.float32) # 创建空图像  
    for i in range(h):  
        for j in range(w):  
            t_img[i,j] = c*math.pow(image[i,j], gamma)  
    cv2.normalize(t_img,t_img,0,255,cv2.NORM_MINMAX) # 像素值归一化0~255  
    return t_img  
  
img = cv2.imread(r'./1.jpg', 0)  
t_img = gamma(1, 0.5, img)  
cv2.imwrite(r'./gamma1.jpg', t_img)  
cv2.waitKey(0)
```

3、结果图



4、结果分析

在本题中，通过使用 $\gamma < 1$ 的伽马变换，对**曝光不足（过暗）**的灰度图进行对比度调节。具体就是，通过**非线性变换**让图像中灰度值过大的区域的灰度值得到降低，从而使得图像中**暗部细节得到增强**，最终图像整体的细节表现得到增强。

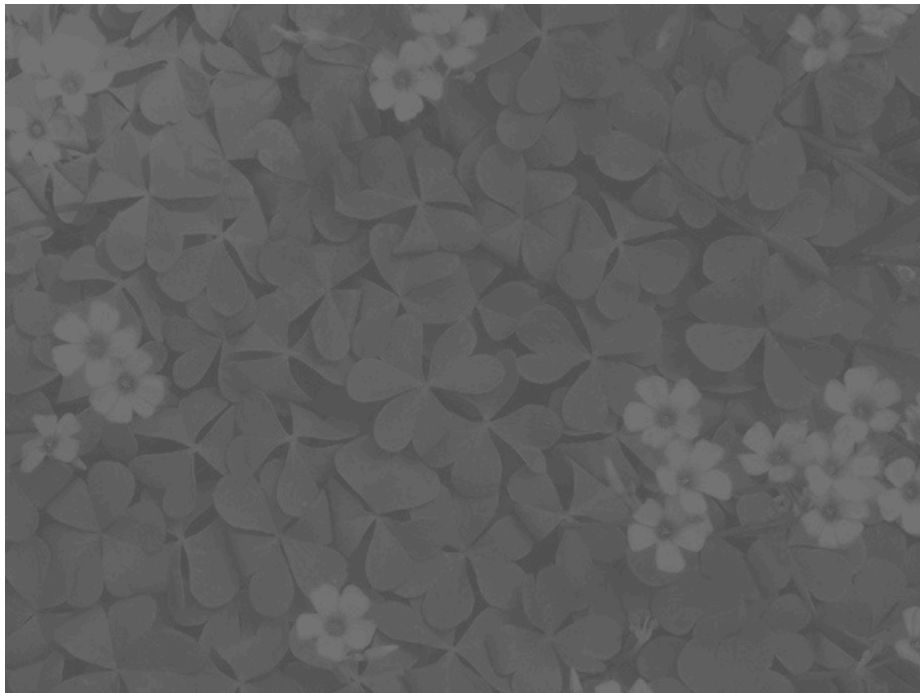
对于Gamma 校正后图像的像素值范围**可能超出255**的问题，使用cv2.NORM_MINMAX将像素值线性映射到0~255区间，确保图像可以正常显示和保存。归一化后，图像的对比度得到保留，同时像素值范围符合标准。

正确答案：

2. (简答题)

(图像增强flower) 请对下图进行图像增强，方法不限。

(内容请包含 1. 算法原理； 2. 关键代码； 3. 结果图； 4. 结果分析)



我的答案：

50 分

1、算法原理

对数变换 (Log Transformation)：通过对图像中的每个像素值进行对数运算，将较暗的像素值映射到更宽的范围，从而增强图像的细节。首先创建一个与输入图像大小相同的空图像，然后遍历输入图像的每

个像素，对其像素值进行对数运算，公式为

$$s = c \cdot \log(1 + r)$$

其中c为常数，用于控制变换的强度。

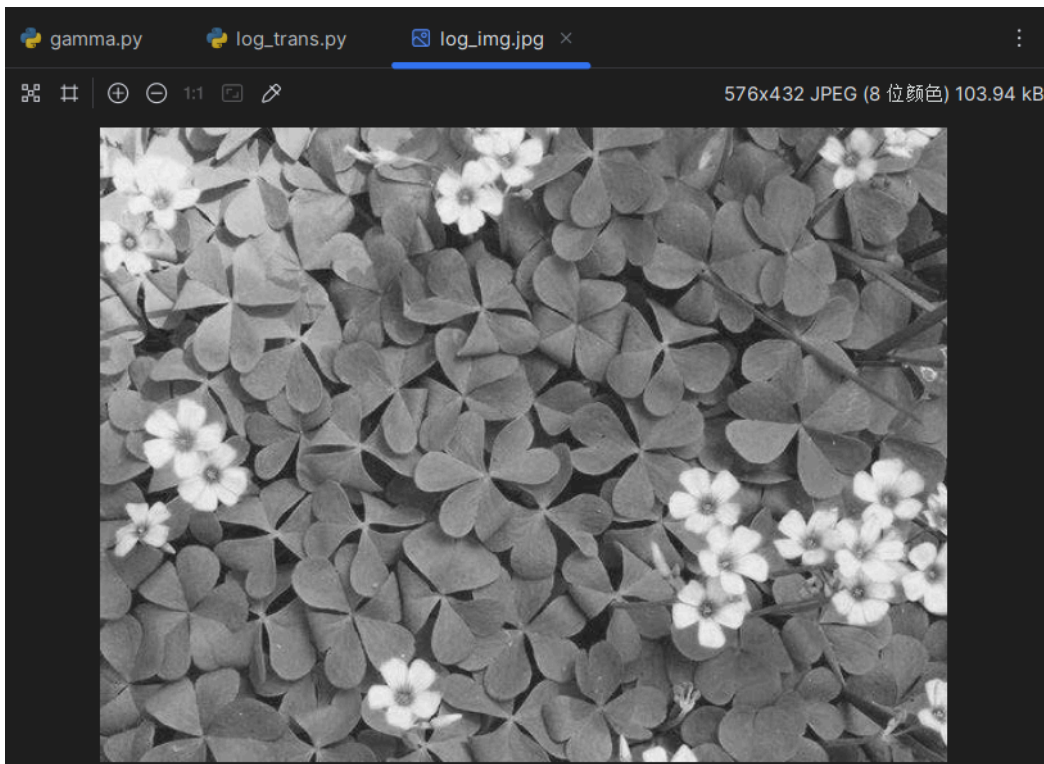
对数变换能够压缩高亮度区域的动态范围，同时扩展低亮度区域的动态范围，使得图像的细节更加清晰。最后，再使用cv2模块normalize函数的NORM_MINMAX方法，将图像的像素值线性地映射到0~255，而不改变图像的相对对比度。

2、关键代码

```
def log_trans(c,image):
    h,w = image.shape[0],image.shape[1]
    t_img = np.zeros((h,w),dtype=np.float32) # 创建空图像
    for i in range(h):
        for j in range(w):
            t_img[i,j] = c*math.log(1+image[i,j])
    cv2.normalize(t_img,t_img,0,255,cv2.NORM_MINMAX) # 像素值归一化0~255
    return t_img
```

```
img = cv2.imread(r'./2.jpg', 0)
t_img = log_trans(1, img)
cv2.imwrite(r'./log_img.jpg', t_img)
cv2.waitKey(0)
```

3、结果图



4、结果分析

对数变换处理后，图像的对比度得到提升，处理后的图像可以明显看到暗部细节的增强，同时图像的亮部区域也不会出现过曝现象。对于低对比度的图像（比如夜间拍摄的照片），这种方式能有效提升图像的可视化效果。

对于对数变换（Log Transformation）后图像的像素值范围可能超出255的问题，使用cv2.NORM_MINMAX将像素值线性映射到0~255区间，确保图像可以正常显示和保存。归一化后，图像的对比度得到保留，同时像素值范围符合标准。

正确答案：

图像增强2-20250313

题量: 3 满分: 100

智能分析

100分

作答时间: 2025-03-14 09:59 至 2025-03-21 11:59

一. 简答题（共3题，100分）

1. (简答题)

附件中的20幅噪声图像中均加入了均值为0，方差为0.03的高斯噪声，请对图像进行去噪增强，使增强效果尽可能接近图1gray.jpg的效果，请上传增强后的效果图，并写出原理。

(内容请包含 1. 算法原理； 2. 关键代码； 3. 结果图； 4. 结果分析)



noise.rar
5.61 MB

我的答案：

33.3 分

1、算法原理

对于一个含有噪声的图像，有：

$$g(x,y) = f(x,y) + n(x,y)$$

其中， $n(x,y)$ 为噪声，且具有零均值（zero average value）。

对M个（本题M=20）不同的噪声图像进行平均，以减少噪声，从而对图像进行去噪增强：

$$\bar{g}(x,y) = \frac{1}{M} \sum_{i=1}^M g_i(x,y) \quad \delta^2_{\bar{g}(x,y)} = \frac{1}{M} \sigma^2_{n(x,y)}$$
$$E[\bar{g}(x,y)] = f(x,y) = E[g(x,y)] \quad \delta_{\bar{g}(x,y)} = \frac{1}{\sqrt{M}} \sigma_{n(x,y)}$$

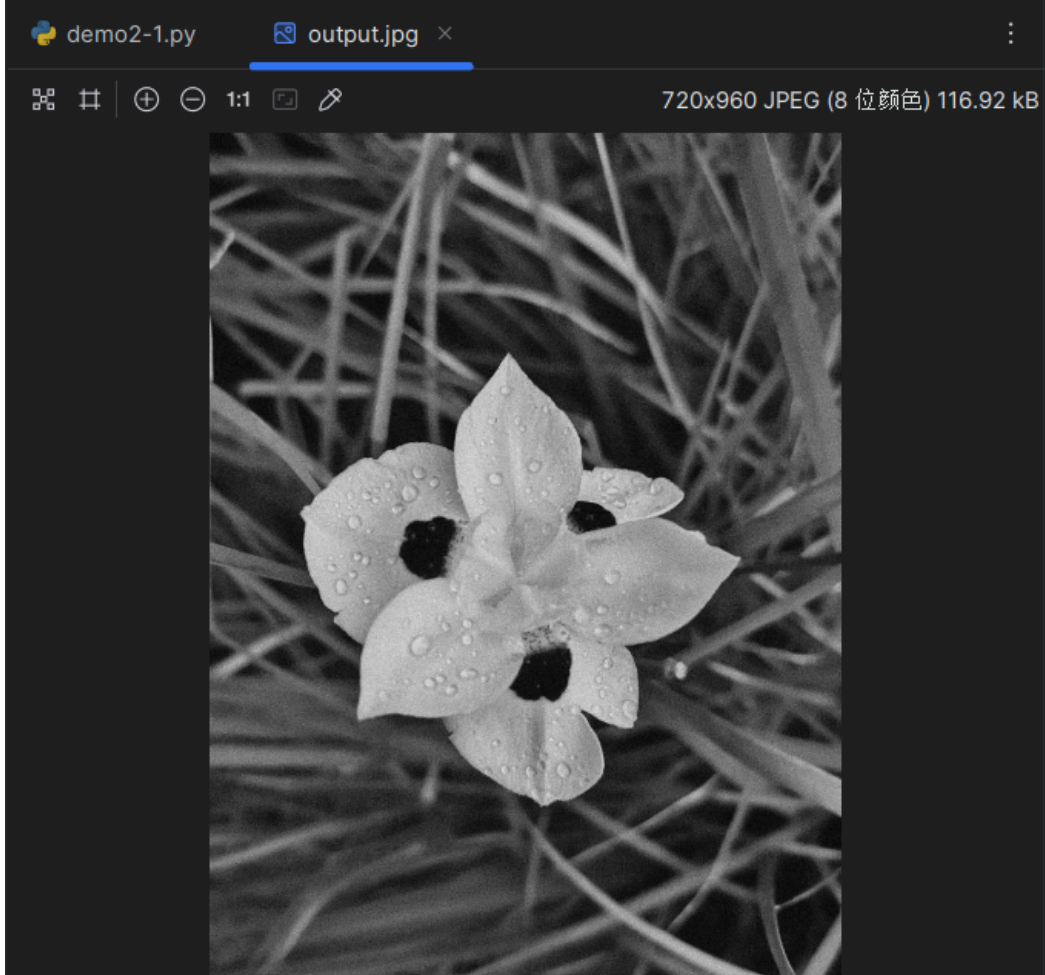
随着M值的增加，每个像素位置的像素值的方差 $\delta^2_{\bar{g}(x,y)}$ 降低。

这意味着：在图像平均中使用的噪声图像数量越多， $g(x,y)$ 越接近于 $f(x,y)$ 。

2、关键代码

```
sum = None
for i in range(1,21):
    img = Image.open(f"./Frame Average/image_noise{i}.jpg")
    img_array = np.array(img, dtype=np.float32)
    if sum is None:
        sum = np.zeros_like(img_array, dtype=np.float32) # 创建全零数组，和输入数组形状、数据类型相同
    sum += img_array
aver_image = np.clip(sum/20,0,255).astype(np.uint8)
Image.fromarray(aver_image).save("output.jpg")
```

3、结果图



4、结果分析

图像平均算法基于统计学的中心极限定理，通过叠加多幅独立同分布噪声的图像实现降噪。

对于均值为0、方差为 σ^2 的高斯噪声，若对M幅图像进行平均，则噪声方差会降低，关系式为：

$$\delta_{\bar{g}(x,y)}^2 = \frac{1}{M} \sigma_{n(x,y)}^2$$

而图像信号因多幅对齐叠加得以保留。处理后图像噪声显著减少，信噪比提升，纹理细节更清晰。

当然受限于有限样本数（M=20），可能存在少量残余低频噪声。

正确答案：

答案解析：

2. (简答题)

请分别用均值滤波器和中值滤波器去除图像中的椒盐噪声，并比较去噪结果和说明原理。要求：卷积代码需自己写（请写出算法原理，关键代码，实验结果和分析）



image_salt_pepper2 (2).zip
94.7 KB

我的答案：

33.3 分

1、算法原理

(1) **均值滤波**：用邻域像素的算术平均值代替中心像素值，消除尖锐噪声，但可能导致边缘模糊。

步骤：使用`np.pad`对图像边缘进行镜像填充，保证窗口滑动时边缘区域有足够数据；通过`sliding_window_view`生成(`ksize*ksize`)大小的局部窗口视图，内存共享避免拷贝；`np.mean`对每个窗口内的所有像素取算术平均；`astype(image.dtype)`将浮点结果强制转换回原数据类型。

(2) **中值滤波**：用邻域像素的中位数代替中心像素值，有效消除孤立噪点，同时保留边缘锐度。

步骤：边缘填充、生成局部窗口同上；`np.median(windows, axis=(2, 3))`对窗口内像素排序后取中间值；`astype(image.dtype)`强制转换回原数据类型。

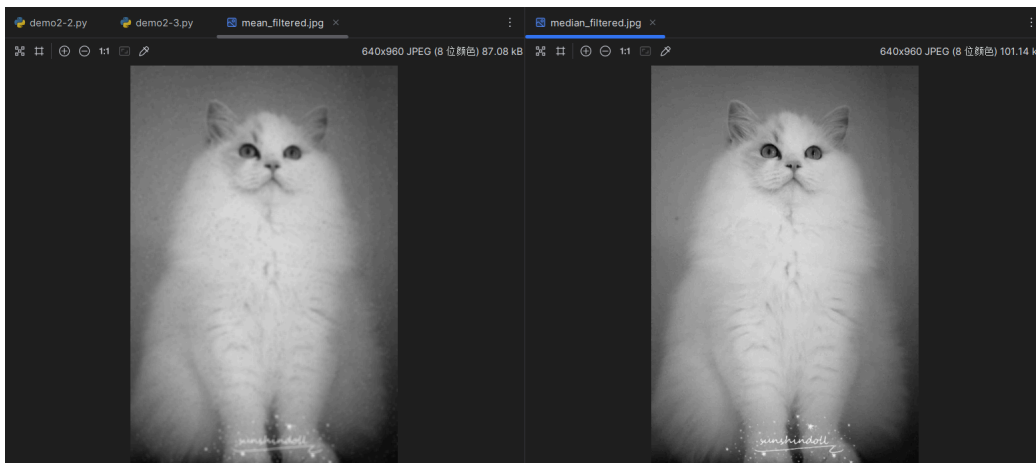
2、关键代码

```
def mean_filter(image, ksize):
    pad = ksize // 2
    padded = np.pad(image, pad, mode='edge')
    windows = sliding_window_view(padded, (ksize, ksize))
    return np.mean(windows, axis=(2, 3)).astype(image.dtype)
```

```
def median_filter(image, ksize):
    pad = ksize // 2
    padded = np.pad(image, pad, mode='edge')
    windows = sliding_window_view(padded, (ksize, ksize))
    return np.median(windows, axis=(2, 3)).astype(image.dtype)
```

```
image = cv2.imread('2-2.jpg', cv2.IMREAD_GRAYSCALE)
mean_filtered = mean_filter(image, ksize=7)
median_filtered = median_filter(image, ksize=3)
```

3、效果图（左图为均值滤波结果，右图为中值滤波结果）



4、实验结果和分析

均值滤波：

基于邻域像素的算术平均运算，通过平滑局部区域抑制高斯噪声，但会因均匀化计算导致图像边缘模糊化。

其计算效率较高，适合处理背景平滑的场景，但输出值可能生成原图中不存在的新灰度值（如均值非整数时）。

当使用较大核（如 7×7 ）覆盖更多像素以稀释噪点影响，反而也导致了图像更加模糊。

中值滤波：

取邻域像素的中位数替代中心像素值，具备抵抗脉冲噪声（如椒盐噪点）的能力，能在消除孤立黑/白点的同时较好保留边缘锐度。

其计算需对窗口内像素排序，效率低于均值滤波，但输出值必为原始像素值之一，避免了人为引入新灰度值。

针对孤立噪点，小尺寸核（如 3×3 ）即可高效清理，且对大面积连续噪声可通过增大核尺寸灵活应对。

对比总结：

去除本题中的椒盐噪声，使用中值滤波（`ksize=3~5`）处理的效果更好。

正确答案：

答案解析：

HE-请对下图进行直方图均衡，以增强图像对比度。（附件中需增强的图为fig1，fig3为原图，fig2-gray是原图的灰度图）



fig1.zip

241.16 KB

33.4 分

我的答案：

1、算法原理

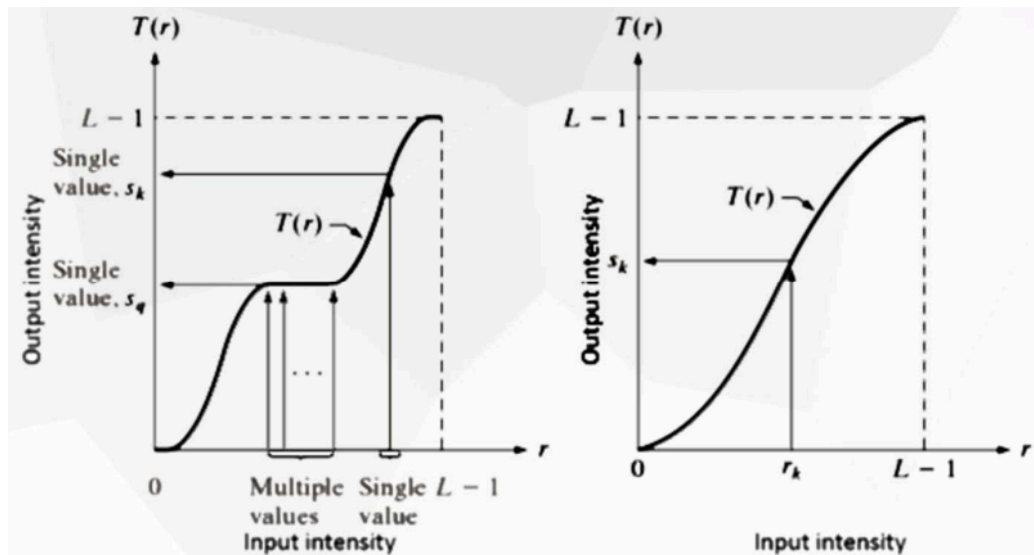
r: 归一化的原图像灰度（归一化至0~1区间）

s: 经直方图均衡化后的图像灰度（应当仍然在0~1区间）

直方图均衡化：根据直方图对像素点的灰度值进行变换。在 $[0, 1]$ 区间内的任何一个 r ，经变换函数 $T(r)$ 都可以产生一个对应的 s ，且

$$s = T(r)$$

T应当满足以下两个条件：

在 $0 \leq r \leq 1$ 内， $T(r)$ 为 **单调递增函数**（保证均衡化后图像的灰度级从黑到白的次序不变）在 $0 \leq r \leq 1$ 内， $0 \leq T(r) \leq 1$ （保证均衡化后图像的像素灰度值在允许的范围内）

- PDF relation between p_s and p_r p_s 与 p_r 的PDF（概率密度分布）关系

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right|$$

- If the cumulative distribution function is used as the transform
若将累积分布函数 (CDF) 用作变换：

$$s = T(r) = (L-1) \int_0^r p_r(\omega) d\omega$$

- then the PDF of the transformed image is
那么变换后图像的概率密度分布（PDF）为：

$$p_s(s) = p_r(r) \left| \frac{dr}{ds} \right| = p_r(r) \left| \frac{1}{(L-1)p_r(r)} \right| = \frac{1}{L-1} \quad 0 < s < L-1$$

2、关键代码

img = Image.open('fig1.jpg').convert('L')

img_array = np.array(img)

计算直方图

hist, bins = np.histogram(img, 256, [0,256])

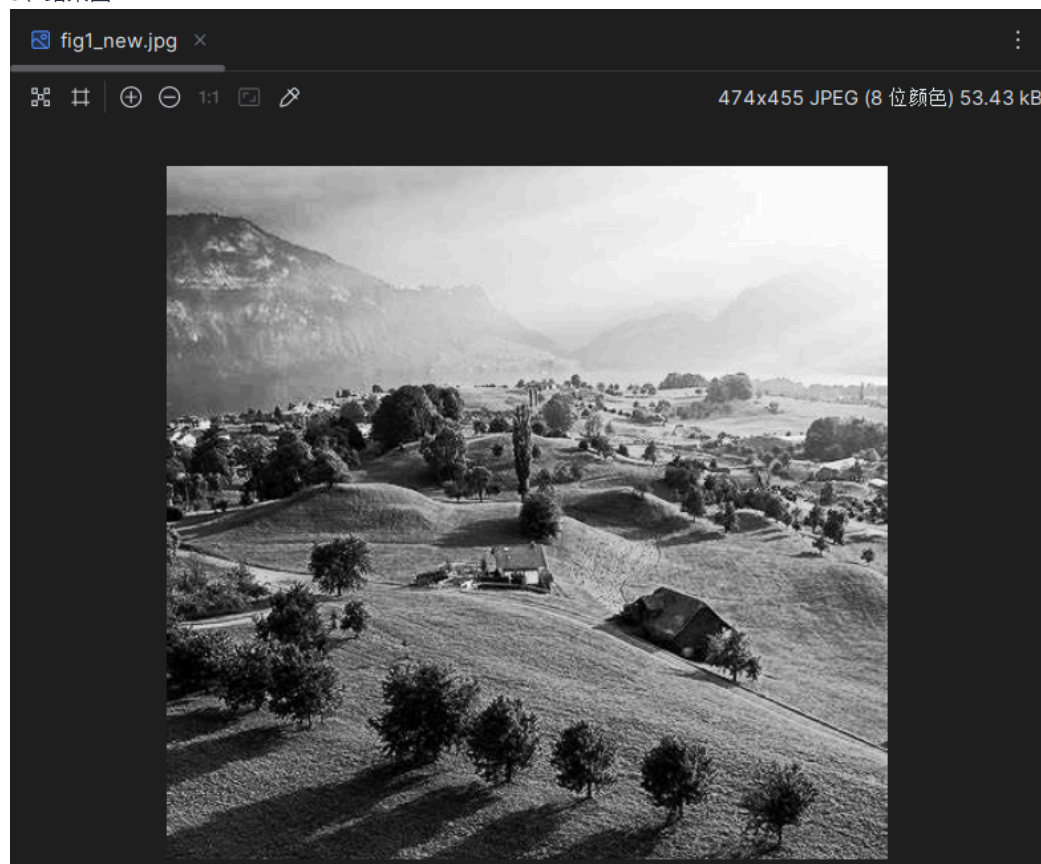
计算累积分布函数 (CDF)

cdf = hist.cumsum()

cdf_normalized = cdf * hist.max() / cdf.max()

```
# 使用累积分布函数 (CDF) 作为变换函数进行直方图均衡化
cdf_m = np.ma.masked_equal(cdf,0)
cdf_m = (cdf_m - cdf_m.min())*255/(cdf_m.max()-cdf_m.min())
cdf = np.ma.filled(cdf_m,0).astype('uint8')
img2 = cdf[img]
Image.fromarray(img2).save('fig1_new.jpg')
```

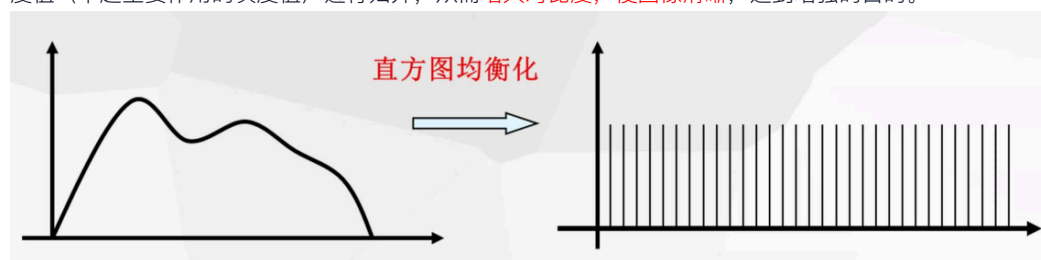
3、结果图



4、结果分析

通过变换函数 $T(r)$ 可以控制图像灰度级的概率密度函数 $p_s(s)$ ，从而改善图像的灰度层次。

直方图均衡化：对在图像中像素个数多的灰度值（起主要作用的灰度值）进行展宽，而对像素个数少的灰度值（不起主要作用的灰度值）进行归并，从而**增大对比度，使图像清晰**，达到增强的目的。



例如本题，fig1.jpg为**过曝光图像**，灰度级集中在高亮度范围内。

采用直方图均衡化，把原始图像的直方图变换为**均匀分布（均衡）**的形式，增加了像素之间灰度值差别的动态范围，从而增强图像整体对比度。

正确答案：

图像增强3-20250320

题量: 2 满分: 100

智能分析

95分

作答时间: 2025-03-21 10:49 至 2025-03-28 11:49

一. 简答题（共2题，100分）

1. (简答题)

请对下图进行锐化处理，并写出所用算子和分析结果
(内容请包含 1. 算法原理； 2. 关键代码； 3. 结果图； 4. 结果分析)



x-rayblog-gaussian blur d4.rar
339.65 KB

我的答案:

48 分

1、算法原理

通过在原始图像上应用**Sobel算子**，增强图像中的水平和垂直边缘。

水平方向卷积核 sobel_x：检测垂直边缘，强调左右像素差异；

垂直方向卷积核 sobel_y：检测水平边缘，强调上下像素差异。

两个卷积核分别表示如下：

$$T_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} \quad T_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

遍历图像每个像素（跳过边缘防止越界），提取3x3邻域。

分别与sobel_x和sobel_y进行**逐元素相乘并求和**，得到水平梯度**Gx**和垂直梯度**Gy**。

计算梯度幅值： $\sqrt{G_x^2 + G_y^2}$ ，存入img_sobel，表示**边缘强度**，值越大边缘越明显。

img1.jpg：将梯度幅值转换为8位无符号整数。由于未归一化，过高的值可能被截断至255，直接显示边缘。

img2.jpg：将原图与边缘强度叠加，np.clip确保像素值在0-255之间。边缘区域变亮，起到锐化效果。

2、关键代码

导入模块，灰度读取转为矩阵，略

h,w = img_arr.shape

img_sobel = np.zeros((h,w))

sobel_x = np.matrix([[[-1,0,1],[-2,0,2],[-1,0,1]]])

sobel_y = np.matrix([[[-1,-2,-1],[0,0,0],[1,2,1]]])

for i in range(1,h-1):

for j in range(1,w-1):

img_sobel[i,j] = math.sqrt((np.sum(np.multiply(img_arr[i-1:i+2,j-1:j+2],sobel_x))) ** 2 +
(np.sum(np.multiply(img_arr[i-1:i+2,j-1:j+2],sobel_y))) ** 2)


```
img1 = np.uint8(img_sobel)
img2 = img_arr + img_sobel
img2 = np.clip(img2,0,255).astype(np.uint8) # 归一化至0~255
# Image模块输出图像， 略
```

3、结果图（分别为"img2.jpg"和"img1.jpg"）



4、结果分析

Sobel算子通过水平（sobel_x）和垂直（sobel_y）两个方向的卷积核，分别计算图像的横向梯度Gx和纵向梯度Gy，计算二者几何平均值梯度幅值表示**边缘强度**生成img1，**将原始的img_array加上img1，得到最终实现了锐化的img2。**

Sobel算子通过**梯度计算**有效提取图像边缘，计算简单、速度快，适合实时或对精度要求不高的场景。然而其结果受噪声影响较大。实际应用中常需结合滤波、阈值分割等方法优化结果。

正确答案：

答案解析：



我的答案：

47 分

1、算法原理

对于二维图像处理，通常使用 x, y 表示离散的空间域坐标变量，用 u, v 表示离散的频率域变量。

二维离散傅里叶变换（DFT）和反变换（IDFT）为：【代码中可直接调用`np.fft.fft2/np.fft.ifft2`函数完成】

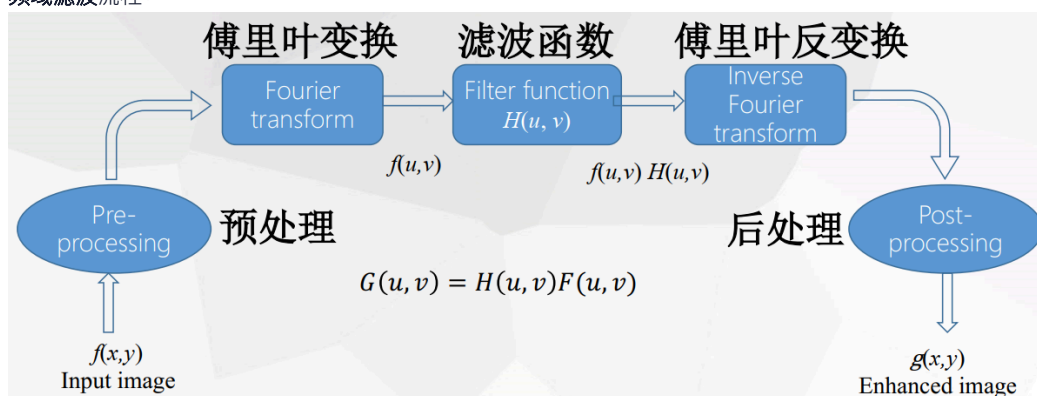
$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(ux/M + vy/N)}$$

二维离散傅里叶变换也可以用极坐标表示：

$$F(u, v) = R(u, v) + jI(u, v) = |F(u, v)| e^{j\phi(u, v)}$$

频域滤波流程：



滤波步骤公式：【注意在滤波之前，必须将传递函数的原点到频率矩形的中心】

- $f(x, y)(-1)^{x+y} = f_{new}(x, y)$
- $F(u - M/2, v - N/2) = \mathcal{F}(f_{new}(x, y))$
- $G(u, v) = H(u, v)F(u - M/2, v - N/2)$
- $G(x, y) = \mathcal{F}^{-1}(G(u, v))$
- $G_{new} = G(x, y)(-1)^{x+y}$

2、关键代码

傅里叶变换并**中心化**

```
fft = np.fft.fft2(img_array)
```

```
fft_shift = np.fft.fftshift(fft)
```

计算**对数幅度谱**（用于观察）

```
magnitude_spectrum = 20 * np.log(np.abs(fft_shift) + 1e-6)
```

图像尺寸参数

```
rows, cols = img_array.shape
```

```
crow, ccol = rows//2, cols//2 # 中心坐标
```

滤波器参数（根据频谱图调整）

```
delta_x = 28 # 条纹频率距中心的水平距离
```

```
w = 18 # 滤波区域宽度
```

```
h = 2 # 滤波区域高度
```

创建全通掩膜

```
mask = np.ones((rows, cols), dtype=np.uint8)
```

在正负频率位置创建矩形滤波区域

```
mask[crow-h:crow+h, ccol+delta_x-w:ccol+delta_x+w] = 0
```

```
mask[crow-h:crow+h, ccol-delta_x-w:ccol-delta_x+w] = 0# 应用掩膜
```

```
fft_shift_filtered = fft_shift * mask
```

逆变换回空间域

```
fft_ishift = np.fft.ifftshift(fft_shift_filtered)
```

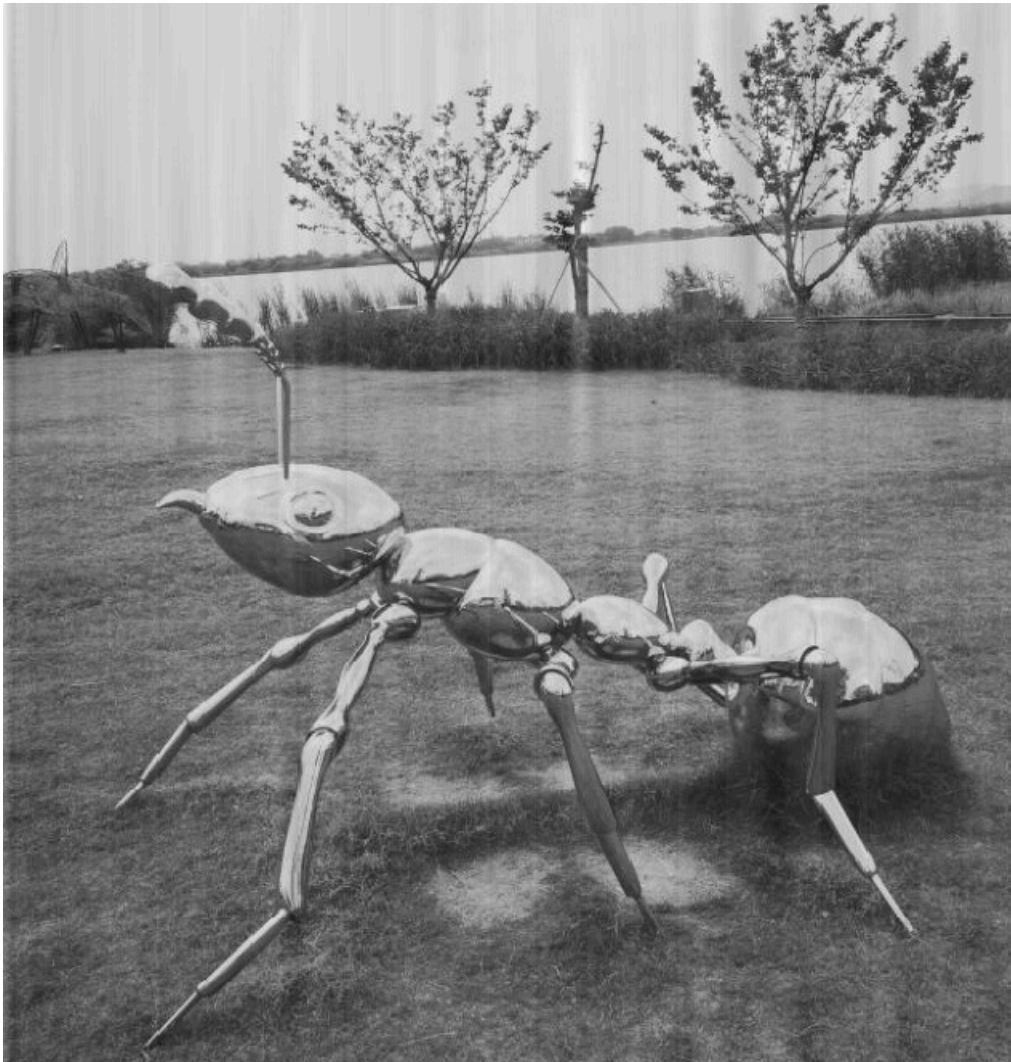
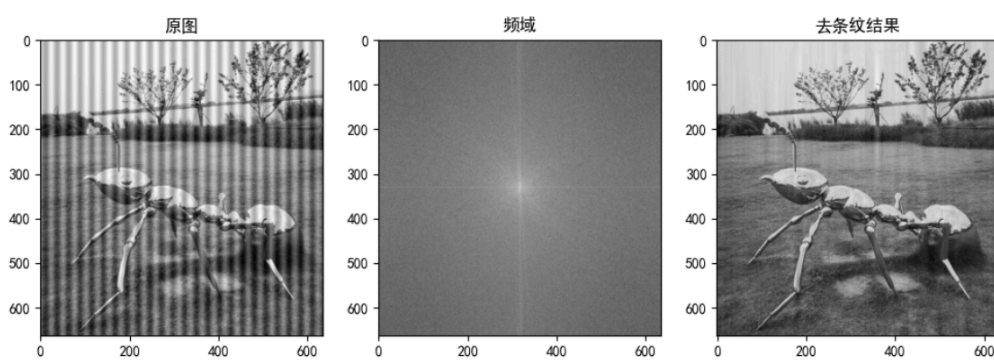
```
img_filtered = np.fft.ifft2(fft_ishift)
```

```
img_filtered = np.abs(img_filtered).astype(np.uint8)
```

```
Image.fromarray(img_filtered).save('result.png')
```

#使用 matplotlib 绘制原图/频谱/变换后图像，略

3、结果图



4、结果分析

傅里叶滤波处理后，原始图像中的等间隔纵向条纹基本被有效去除。

残留轻微**振铃效应**（边缘**弱伪影**），由矩形滤波器陡峭截断高频分量导致，可用**高斯衰减滤波器**优化。

本题取 $\Delta x=28$ ，覆盖了条纹基频分量，基本实现滤波功能。

正确答案：

同态+彩色20250327

题量: 2 满分: 100

智能分析

92分

作答时间: 2025-03-28 09:58 至 2025-04-04 11:58

一. 简答题 (共2题, 100分)

1. (简答题)

(Color image)

Consider the following 500×500 RGB image, in which the squares are fully saturated red, green, and blue, and each of the colors is at maximum intensity. An HSI image is generated from this image.

(a) The **hue** component of the HIS image is smoothed using an averaging mask of size 25×25 . Describe the appearance of the result (you may ignore image border effects in the filtering operation).

(b) Repeat (b) for the **saturation** image.



47 分

我的答案:

1、算法原理

在HSI颜色空间中, 颜色由色相、饱和度和强度三个分量表示。

H (色相): 颜色在色轮上的位置, 取值 $0 \sim 360^\circ$, 从红开始逆时针旋转, 类似于彩虹的环。

S (饱和度): 颜色的纯度或鲜艳程度, 取值 $0 \sim 1$ 。值为0表示灰色或无色的, 值为1表示颜色非常纯。

I (强度): 表示颜色的亮度或明度, 取值 $0 \sim 1$ 。值为0表示黑色, 值为1表示白色。

(1) 给定一幅RGB彩色图像, 每个像素的H,S,I分量计算方式如下:

RGB values have been normalized to the range $[0, 1]$ RGB值已归一化至 $[0, 1]$

$$H = \begin{cases} \theta & \text{if } B \leq G \\ 360 - \theta & \text{if } B > G \end{cases}$$

with

$$\theta = \cos^{-1} \left\{ \frac{\frac{1}{2}[(R-G) + (R-B)]}{[(R-G)^2 + (R-B)(G-B)]^{1/2}} \right\}$$

Hue can be normalized to the range $[0, 1]$ by dividing by 360° .

色调可以通过除以 360° 归一化至 $[0, 1]$

The saturation component S is given by 饱和度分量S的公式

$$S = 1 - \frac{3}{(R+G+B)} [\min(R, G, B)]$$

The intensity component I is defined by 强度分量I定义为

$$I = \frac{1}{3}(R+G+B)$$

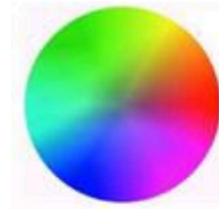
(2) 给定一幅HSI彩色图像, 每个像素的R,G,B分量计算方式如下:

➤ *RG sector* ($0^\circ \leq H < 120^\circ$):

$$B = I(1 - S)$$

$$R = I \left[1 + \frac{S \cos(H)}{\cos(60^\circ - H)} \right]$$

$$G = 3I - (R + B)$$



➤ *GB sector* ($120^\circ \leq H < 240^\circ$): first subtract 120° from H , then

$$H = H - 120^\circ \quad R = I(1 - S)$$

$$G = I \left[1 + \frac{S \cos(H)}{\cos(60^\circ - H)} \right]$$

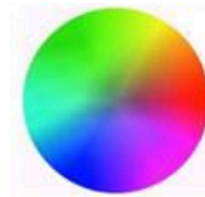
$$B = 3I - (R + G)$$

➤ *BR sector* ($240^\circ \leq H < 360^\circ$): first subtract 240° from H , then

$$H = H - 240^\circ \quad G = I(1 - S)$$

$$B = I \left[1 + \frac{S \cos(H)}{\cos(60^\circ - H)} \right]$$

$$R = 3I - (G + B)$$



2、代码

(a) 对H分量的变换

import cv2

import numpy as np

def Calculate_H(R, G, B):

 a = 0.5 * ((R - G) + (R - B))

 b = np.sqrt((R - G) ** 2 + (R - B) * (G - B)) + 1e-6

 theta = np.arccos(a/b)

 H = np.degrees(theta) / 360.0

 np.putmask(H, B > G, (360 - np.degrees(theta)) / 360.0)

 H[b <= 1e-6] = 0

分母为极小值，还原为0

 return H

def Calculate_S(R, G, B):

 I = (R + G + B) / 3.0

 min_val = np.minimum(np.minimum(R, G), B)

 return np.where(I > 1e-6, 1 - min_val / (I + 1e-6), 0) # 防止分母为0

def RGB2HSI(rgb_img):

 B, G, R = cv2.split(rgb_img/255.0)

 H = Calculate_H(R, G, B)

 S = Calculate_S(R, G, B)

 I = (R + G + B) / 3.0

 # 合并通道

 return np.clip(cv2.merge([H*255, S*255, I*255]), 0, 255).astype(np.uint8)

def HSI2RGB(hsi_img):

 H, S, I = cv2.split(hsi_img/255.0)

 H = H * 360

转换为角度制

 sec = np.floor(H/120).astype(int)%3

分段，sec取0,1,2

 H = H - 120*sec

sec取1/2分别先减掉120/240

 cos_H = np.cos(np.radians(H))

 cos_60_H = np.cos(np.radians(60-H))

$Z = I * (1 + S * \cos_H / \text{np.maximum}(\cos_{60_H}, 1e-6))$ # 防止分母为0

初始化RGB通道

R = np.zeros_like(I)

G = np.zeros_like(I)

B = np.zeros_like(I)

扇区0: 0°-120°

mask0 = (sec == 0)

R[mask0] = Z[mask0]

B[mask0] = I[mask0] * (1 - S[mask0])

G[mask0] = 3 * I[mask0] - (R[mask0] + B[mask0])

扇区1: 120°-240°

mask1 = (sec == 1)

G[mask1] = Z[mask1]

R[mask1] = I[mask1] * (1 - S[mask1])

B[mask1] = 3 * I[mask1] - (R[mask1] + G[mask1])

扇区2: 240°-360°

mask2 = (sec == 2)

B[mask2] = Z[mask2]

G[mask2] = I[mask2] * (1 - S[mask2])

R[mask2] = 3 * I[mask2] - (G[mask2] + B[mask2])

合并通道, 并转换回0-255范围

rgb_img = np.clip(cv2.merge([B * 255, G * 255, R * 255]), 0, 255).astype(np.uint8)

return rgb_img

def average_filter(hsi_img, d):

hsi_img[:, :, d] = cv2.blur(hsi_img[:, :, d], (25, 25))

return hsi_img

rgb_img = cv2.imread('image1.bmp', cv2.IMREAD_COLOR)

hsi_img = RGB2HSI(rgb_img)

hsi_img_blur = average_filter(hsi_img, 0) # d=0为第一维, 即H

rgb_img2 = HSI2RGB(hsi_img_blur)

cv2.imwrite("HSI.jpeg", hsi_img)

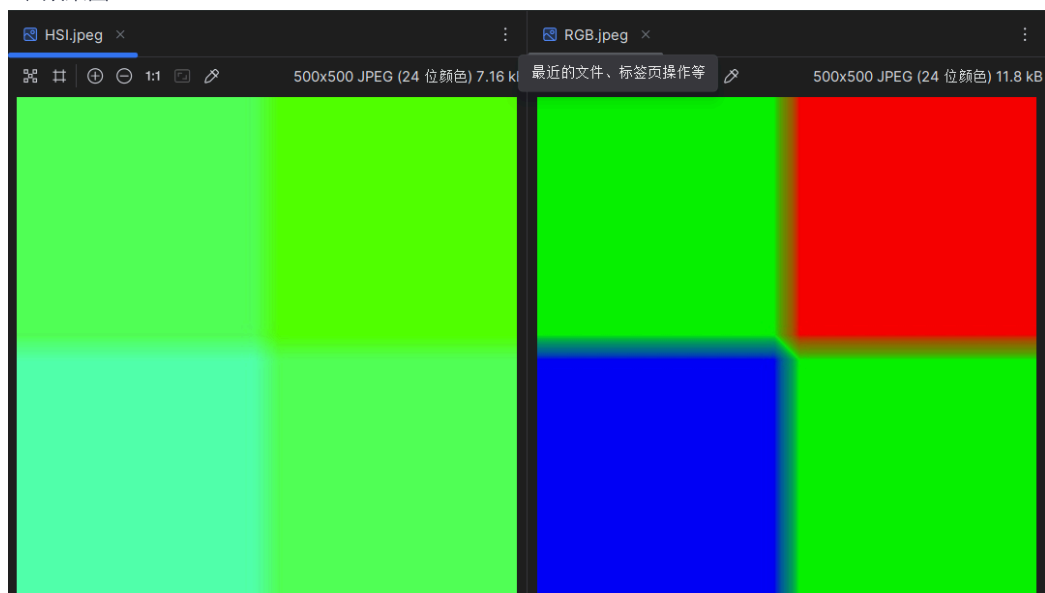
cv2.imwrite("RGB.jpeg", rgb_img2)

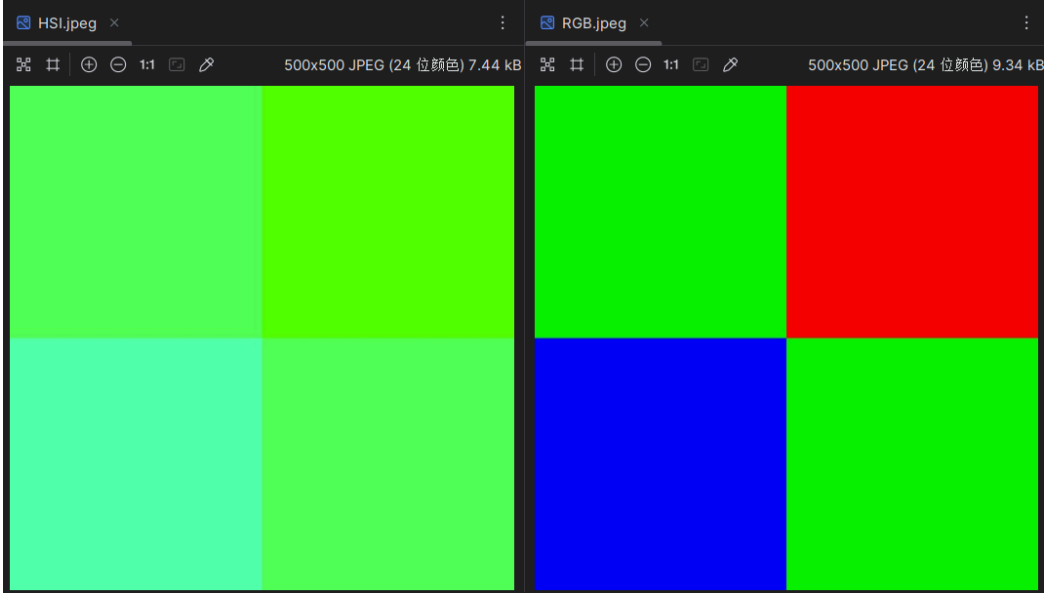
cv2.waitKey(0)

(b) 对S分量的变换

标红代码改为: `hsi_img_blur = average_filter(hsi_img, 1)` # d=1为第二维, 即S

3、效果图





4、结果分析

(a) 将图像转换到HSI空间，对**H分量**图像用一个25x25的算术平均掩模进行处理，再转换回到RGB空间。

交界处产生约为**25像素的模糊**：代码中的`cv2.blur(hsi_img[:, :, 0], (25, 25))`对H通道执行 25×25 的均值模糊，导致色相在边界处混合扩散。由于相邻区域的H值会相互影响，因此边界处颜色会**逐渐过渡**，而不是突变。

(b) 对**S分量**作同上处理。

几乎与原图没有差异。S不影响色调H和亮度I，即颜色本身的位置不会改变。

如果S仍然接近1（大部分区域仍是高饱和度），那么转换回RGB后几乎不会察觉到变化。

即使S变小一点，也只会稍微降低颜色的鲜艳度，而不会导致明显的边界模糊。

正确答案：

2. (简答题)

请对下图进行增强处理，将屋内细节显示出来。（包括算法介绍，代码，结果图和结果分析）



我的答案：

1、算法介绍

45 分

(1) 同态滤波反射模型

一副图像可以表示为其**照度(illumination)**分量 $n(x, y)$ 和**反射(reflectance)**分量 的乘积，虽然在时域上这两者是不可分离的，但是经由**傅立叶变换**两者在频域中可以线性分离。

照度可视为环境中的照明，相对变化很小，可以看作是图像的低频成分；

反射率相对变化较大，则可视为高频成分。

同态滤波利用去除**乘性噪声(multiplicative noise)**，同时增加了对比度以及标准化亮度，达到图像增强的目的。通过分别处理照度和反射率对像素灰度值的影响，通常是借由**高通滤波器(high-pass filter)**，让图像的照明更加均匀，达到增强阴影区细节特征的目的。



(2) 同态滤波算法基本步骤



(3) 参数 的选择对于滤波效果的影响



2、代码

```
import cv2
```

```
import numpy as np
```

```
def homo(img, d0=10, rl=0.5, rh=2.5, c=2, h=2.0, l=0.5):
    img_log = np.log(img + 1e-5) # 加上极小数防止 log(0) 报错
    rows, cols = img_log.shape
    Z = np.fft.fft2(img_log)      # 傅里叶变换 (FFT)
    Z = np.fft.fftshift(Z)        # FFT 中心化
    M, N = np.meshgrid(np.arange(-cols // 2, cols // 2), np.arange(-rows // 2, rows // 2))
    D = np.sqrt(M ** 2 + N ** 2)  # 计算欧氏距离

    # 频域滤波器 H(u,v)
    H = (rh - rl) * (1 - np.exp(-c * (D ** 2 / d0 ** 2))) + rl
    S = H * Z
    S = (h - l) * S + l           # 线性缩放并平移，控制滤波器的斜率

    img_s = np.fft.ifftshift(S)   # IFFT 逆中心化，撤销 fftshift 操作
    img_s = np.fft.ifft2(img_s)   # 傅里叶反变换 (IFFT)
    img_g = np.real(img_s)        # IFFT 取实部

    img_g = np.exp(img_g) - 1      # 还原
    img_g = np.uint8(np.clip(img_g, 0, 255))
    return img_g
```

```
img = cv2.imread('2.png', cv2.IMREAD_GRAYSCALE)
```

```
img_new = homo(img)
```

```
cv2.imwrite("img_new.png", img_new)
```

```
cv2.waitKey(0)
```

3、结果图



4、结果分析

本题使用同态滤波算法，利用去除**乘性噪声(multiplicative noise)**，同时增加了对比度以及标准化亮度，达到图像增强的目的。

滤波后的图像，增强了图片展示的房屋内部细节，包括上方的牌匾以及立柱上的牌匾。

正确答案：

彩色图像分割20250403

题量: 1 满分: 100

智能分析

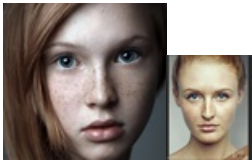
92分

作答时间: 2025-04-15 10:06 至 2025-04-19 10:06

一. 简答题（共1题，100分）

1. (简答题)

请淡化彩色照片中人面部的雀斑，同时尽量保持发丝和睫毛的丝丝顺滑。(内容包括：算法说明，代码，结果图，实验分析)



美颜.zip
237.68 KB

我的答案：

92 分

1、算法说明：

目标是去除面部图像中的雀斑，同时保持眼睫毛、嘴唇等面部细节的清晰度。采用**双边滤波技术**能够在平滑图像时保留边缘信息。通过面部区域的检测与掩膜，对非眼部区域进行滤波，以避免损伤眼部细节。

(1) 面部检测

利用 OpenCV 的 **Haar Cascade 分类器**进行面部检测，通过 **cv2.CascadeClassifier()** 加载训练好的分类器模型，调用 **detectMultiScale()** 方法进行面部识别。

```
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
```

该方法返回面部区域的坐标 (x, y, w, h) ，表示人脸在图像中的位置和尺寸。

(2) 眼睛区域检测

进一步在面部区域内检测眼睛。通过加载眼睛分类器并再次使用 **detectMultiScale()** 方法实现。检测到的眼睛区域将会在滤波过程中保持清晰。眼睛的区域会被单独标记，并且在后续处理时不会受到双边滤波的影响。

```
eyes = eye_cascade.detectMultiScale(roi_gray)
```

(3) 双边滤波

双边滤波通过平滑图像中的色彩信息，同时保持图像的边缘信息，避免去除眼睫毛等细节。使用 OpenCV 的 **cv2.bilateralFilter()** 函数进行双边滤波。

```
bilateral_filtered = cv2.bilateralFilter(image, d=15, sigmaColor=75, sigmaSpace=75)
```

d: 邻域的直径，表示用于滤波的像素区域大小。

sigmaColor: 控制颜色空间的影响，值越大，颜色差异越能影响滤波。

sigmaSpace: 控制坐标空间的影响，值越大，滤波的范围越广。

(4) 掩膜操作

创建一个**掩膜 (mask)**，其中眼睛区域设置为 255（白色），其他区域为 0（黑色）。掩膜用于从原图中提取出眼睛区域，并将其直接复制到滤波后的图像中，以**确保眼睛细节不受影响**。

```
eyes_only_mask[y + ey:y + ey + eh, x + ex:x + ex + ew] = 255
```

(5) 最终图像：双边滤波处理结果和眼睛区域细节之间的合成。

2、代码：

```
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文显示

# 加载图像
image = cv2.imread("2.png")

# 将图像从BGR转为RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# 使用Haar Cascade分类器加载面部特征检测器
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
eye_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')

# 将图像转换为灰度图
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# 检测脸部
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

# 创建一个全黑的掩膜
eyes_only_mask = np.zeros_like(image)

# 提取面部区域并标记眼睛区域
for (x, y, w, h) in faces:
    roi_gray = gray[y:y + h, x:x + w]
    eyes = eye_cascade.detectMultiScale(roi_gray)
    # 在眼睛区域保留细节！！
    for (ex, ey, ew, eh) in eyes:
        eyes_only_mask[y + ey:y + ey + eh, x + ex:x + ex + ew] = 255 # 保留眼睛区域

# 对图像应用双边滤波
bilateral_filtered = cv2.bilateralFilter(image, d=15, sigmaColor=75, sigmaSpace=75)

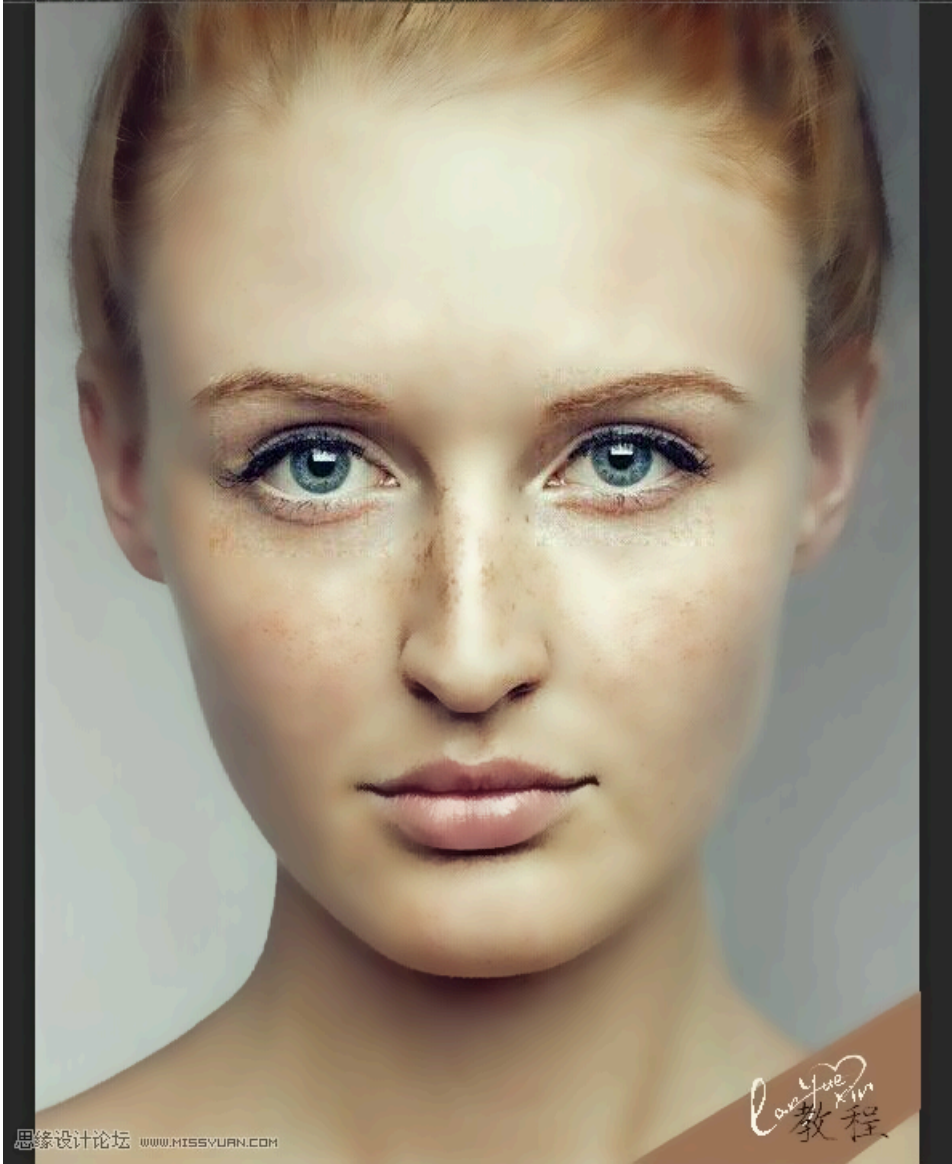
# 将眼睛区域从原图复制到处理后的图像
final_image = np.copy(bilateral_filtered)
final_image[eyes_only_mask == 255] = image[eyes_only_mask == 255]

# 显示并保存处理后的图像
cv2.imshow("Processed Image", final_image)
cv2.imwrite("processed_image.png", final_image)

# 等待用户按键然后关闭窗口
cv2.waitKey(0)
cv2.destroyAllWindows()

```

3、结果图：



4、实验分析：

本算法通过以下几个步骤实现了对面部图像中雀斑的去除，同时保持了面部细节的清晰度：

- (1) 通过面部和眼睛检测定位关键区域。
- (2) 对非关键区域应用**双边滤波**来去除雀斑。
- (3) 使用**掩膜**确保眼睛区域在滤波过程中不受影响。
- (4) **合成**处理图像并保存输出。

通过这种方法，可以有效地去除面部雀斑，同时**保留睫毛毛、嘴唇等细节**，达到高质量的图像修复效果。

正确答案：

图像分割2-20250410

题量: 1 满分: 100

智能分析

100分

作答时间: 2025-04-18 11:40 至 2025-04-25 11:40

一. 简答题（共1题，100分）

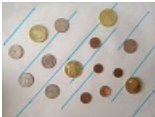
1. (简答题)

请检测出下图中的不同大小的硬币数目与线条数



2-2.rar

83.35 KB



我的答案:

100 分

1、算法原理

采用Canny边缘检测算法，提取图像中的有效边缘信息；采用霍夫变换（Hough Transform），进一步实现对直线和圆形等几何结构的鲁棒检测。

（一）Canny边缘检测

Canny是一种多阶段的边缘检测算法，具有良好的抗噪能力与边缘定位精度。其主要步骤如下：

- （1）高斯滤波（Gaussian Blur）：对图像进行平滑处理，以降低图像中的随机噪声。
- （2）梯度计算（Sobel）：利用Sobel算子分别计算图像在水平方向（Gx）和垂直方向（Gy）上的梯度变化，进而获得每个像素的梯度幅值和方向。
- （3）非极大值抑制（NMS）：沿梯度方向对边缘图像进行抑制处理，保留局部极大值，从而获得更细、更准确的边缘线条。
- （4）双阈值处理与边缘连接（Hysteresis）：设定高低两个阈值——高于高阈值的像素被视为强边缘；低于低阈值的被舍弃；介于两者之间的像素，若与强边缘相连，则保留。

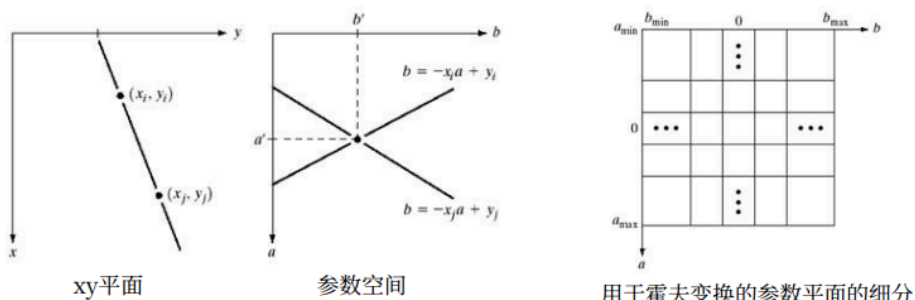
（二）霍夫变换（Hough Transform）——圆

霍夫变换----直线

> 给定 n 个图像中的点， 尝试找到位于直线上的这些点的子集。

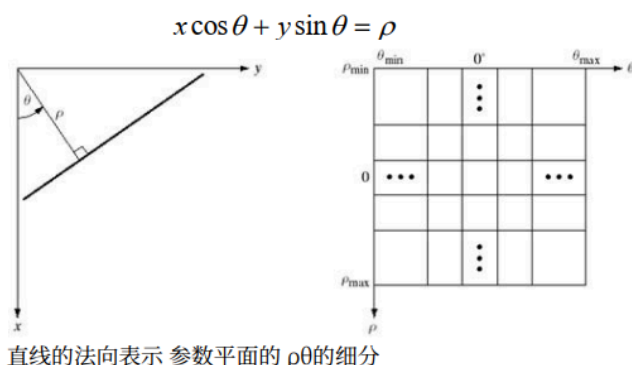
> 考虑一个点 (x_i, y_i) ， 一条以斜截式形式通过它的直线：

$$y_i = ax_i + b \Rightarrow b = -x_i a + y_i$$



一般霍夫变换

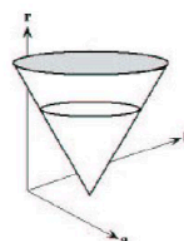
为避免当直线接近平行时斜率 (a) 趋于无穷大，使用直线的法向表示：



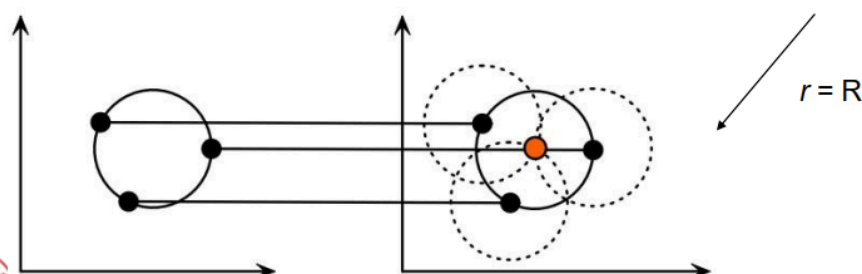
函数的轨迹存在于 $\rho\theta$ -平面中的正弦曲线。霍夫变换适用于任何形式为 $g(\mathbf{v}, \mathbf{c})=0$ 的函数，其中 $\mathbf{v}$ 是坐标向量， $\mathbf{c}$ 是系数向量。

霍夫变换----圆

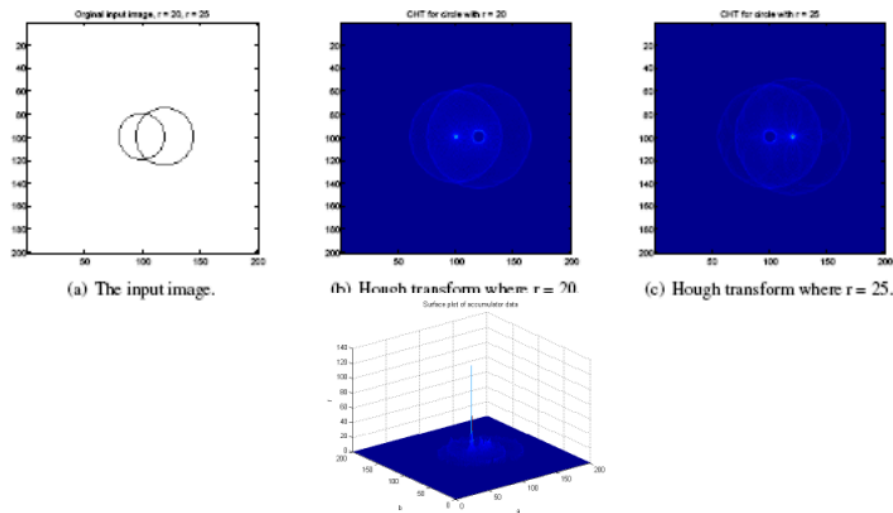
$$r^2 = (x-a)^2 + (y-b)^2 \longrightarrow \begin{aligned} x &= a + r \cos(\theta) \\ y &= b + r \sin(\theta) \end{aligned}$$



参数空间



霍夫变换 ---- 圆



(a) The input image.

(b) Hough transform where $r = 20$.

(c) Hough transform where $r = 25$.

ab平面的表面图, $r = 20$



2、代码

'''硬币检测'''

import cv2

import numpy as np

from sklearn.cluster import KMeans

```
def detect_and_draw_coins_v2(image_path, output_path="2.png"):
```

```
    img = cv2.imread(image_path)
```

```
    out_img = img.copy()
```

```
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
    blurred = cv2.GaussianBlur(gray, (5, 5), 1)
```

```
    edges = cv2.Canny(blurred, 80, 150)
```

```
# Hough 圆检测 (调整参数以减少误检)
```

```
circles = cv2.HoughCircles(
```

```
    blurred,
```

```
    cv2.HOUGH_GRADIENT,
```

```
    dp=1.2,
```

```
    minDist=30,
```

```
    param1=100,
```

```
    param2=35, # 更严格的圆形识别
```

```
    minRadius=20,
```

```
    maxRadius=60
```

```
)
```

```
coin_counts = {}
```

```
if circles is not None:
```

```
    circles = np.uint16(np.around(circles[0]))
```

```
# 过滤: 基于边缘密度
```

```
valid_circles = []
```

```
for x, y, r in circles:
```

```
    mask = np.zeros_like(gray)
```

```
    cv2.circle(mask, (x, y), r, 255, 2)
```

```
    edge_pixels = cv2.countNonZero(cv2.bitwise_and(edges, edges, mask=mask))
```

```
    edge_density = edge_pixels / (2 * np.pi * r)
```

```
    if edge_density > 0.4: # 调整此阈值过滤误圈
```

```
        valid_circles.append((x, y, r))
```

```

if not valid_circles:
    print("未检测到有效的硬币圆形。")
    return

# KMeans 聚类半径
radii = [r for (_, r) in valid_circles]
radii_np = np.array(radii).reshape(-1, 1)
kmeans = KMeans(n_clusters=4, random_state=0).fit(radii_np)
labels = kmeans.labels_
centers = kmeans.cluster_centers_.flatten()

# 标签映射 (小→大)
sorted_idx = np.argsort(centers)
label_remap = np.zeros_like(sorted_idx)
for new_label, old_label in enumerate(sorted_idx):
    label_remap[old_label] = new_label
final_labels = [label_remap[l] for l in labels]

# 彩色标注
color_palette = {
    0: (0, 0, 255), # 红
    1: (0, 255, 0), # 绿
    2: (255, 0, 0), # 蓝
    3: (0, 255, 255) # 黄
}

# 绘图与统计
for idx, (x, y, r) in enumerate(valid_circles):
    lbl = final_labels[idx]
    color = color_palette.get(lbl, (255, 255, 255))
    cv2.circle(out_img, (x, y), r, color, 2)
    cv2.circle(out_img, (x, y), 2, color, 3)
    coin_counts[lbl] = coin_counts.get(lbl, 0) + 1

else:
    print("未检测到任何圆。")

cv2.imwrite(output_path, out_img)
print("最终有效硬币分类及个数：")
for lbl in sorted(coin_counts.keys()):
    print(f"类别 {lbl} : {coin_counts[lbl]} 个")
print(f"输出图像已保存为： {output_path}")

if __name__ == "__main__":
    detect_and_draw_coins_v2("1.png")

'''线条检测'''
import cv2
import numpy as np
from sklearn.cluster import DBSCAN

def draw_line_rho_theta(img, rho, theta, color=(0, 0, 255), thickness=2):
    """根据 rho 和 theta 绘制整条直线（长度足够长）"""
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a * rho
    y0 = b * rho
    # 画一条足够长的直线（跨越图像）
    x1 = int(x0 + 1000 * (-b))
    y1 = int(y0 + 1000 * (a))
    x2 = int(x0 - 1000 * (-b))

```

```

y2 = int(y0 - 1000 * (a))
cv2.line(img, (x1, y1), (x2, y2), color, thickness)

def detect_blue_lines(image_path, output_path="3.png"):
    # 读取图像
    img = cv2.imread(image_path)
    out_img = img.copy()

    # 提取蓝色区域
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    lower_blue = np.array([90, 50, 50])
    upper_blue = np.array([130, 255, 255])
    mask = cv2.inRange(hsv, lower_blue, upper_blue)

    # 边缘检测
    edges = cv2.Canny(mask, 50, 150)

    # Hough Transform (标准版本)
    lines = cv2.HoughLines(edges, 1, np.pi / 180, threshold=100)

    if lines is not None:
        # 提取 rho 和 theta
        features = np.array([[l[0][0], l[0][1]] for l in lines])

        # 归一化角度在 [0,  $\pi$ )
        features[:, 1] = features[:, 1] % np.pi

        # 角度和位置缩放后聚类
        angle_eps = np.deg2rad(3)
        rho_eps = 15
        scaled = np.copy(features)
        scaled[:, 0] /= rho_eps
        scaled[:, 1] /= angle_eps

        # 聚类合并重复线
        db = DBSCAN(eps=1.2, min_samples=1).fit(scaled)
        labels = db.labels_

        n_lines = len(np.unique(labels))
        print(f"检测到蓝色线条: {n_lines}")

        # 为每个类别绘制一条线 (用平均  $\rho$ 、 $\theta$ )
        for label in np.unique(labels):
            group = features[labels == label]
            mean_rho = np.mean(group[:, 0])
            mean_theta = np.mean(group[:, 1])
            draw_line_rho_theta(out_img, mean_rho, mean_theta)

    else:
        print("未检测到蓝色线条。")
        n_lines = 0

    # 保存图像
    cv2.imwrite(output_path, out_img)
    print(f"输出图像已保存为: {output_path}")
    return n_lines

if __name__ == "__main__":
    detect_blue_lines("1.png")

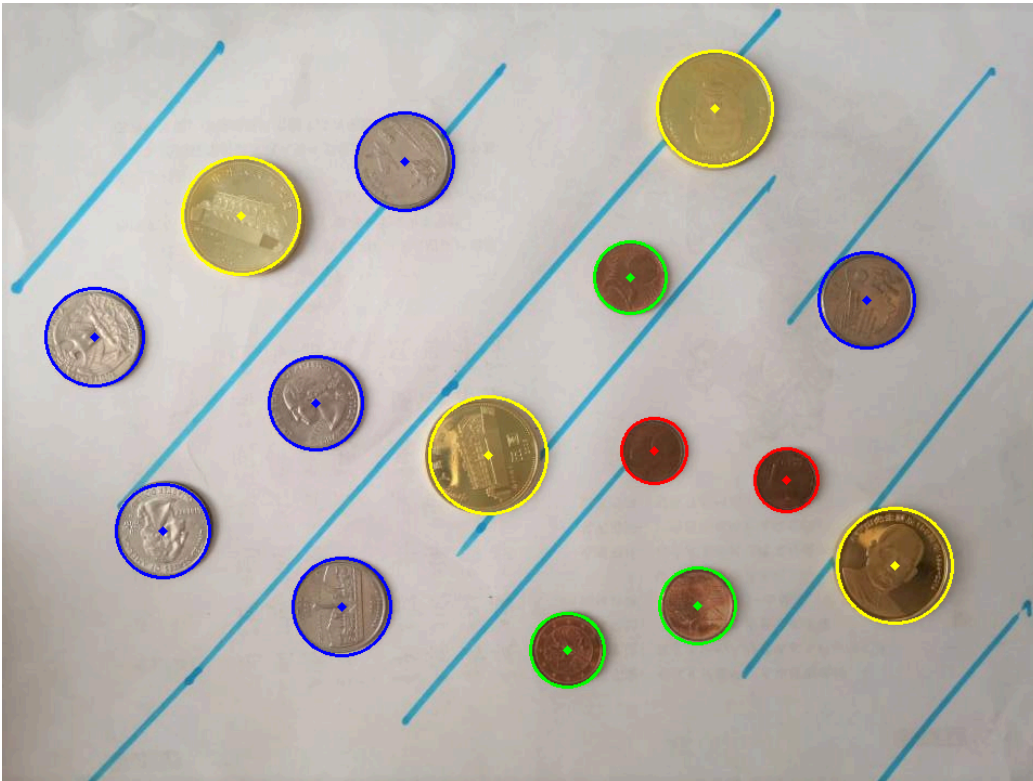
```

3、运行结果

最终有效硬币分类及个数：

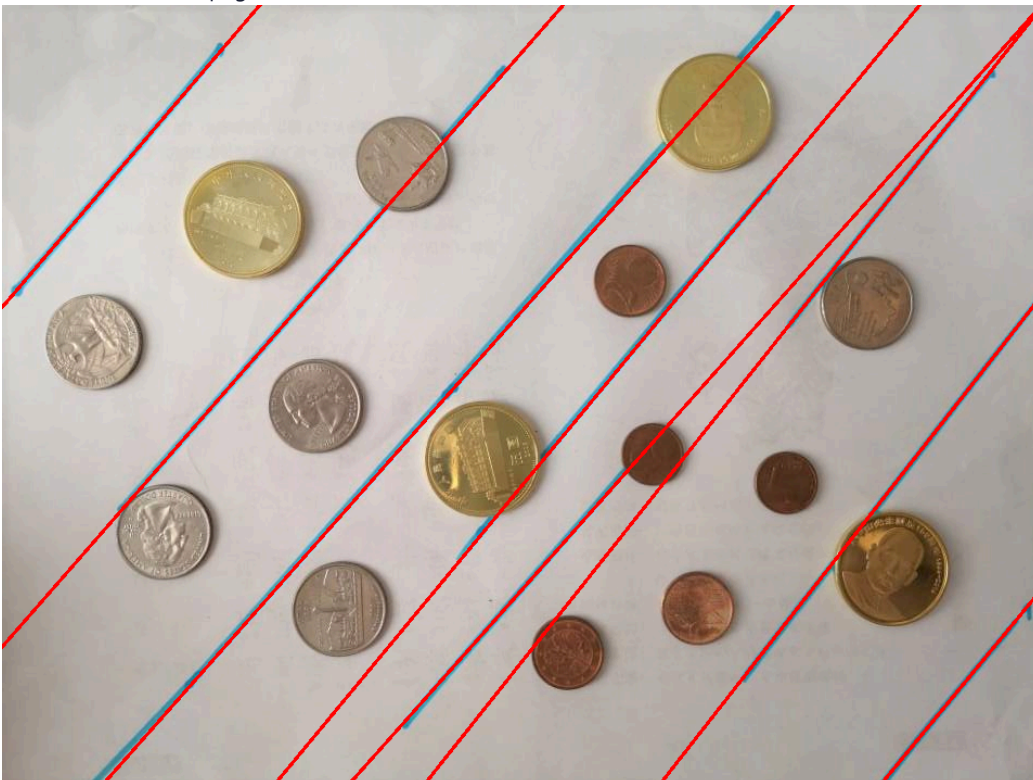
类别 0：2 个
类别 1：3 个
类别 2：6 个
类别 3：4 个

输出图像已保存为：2.png



检测到蓝色线条：8

输出图像已保存为：3.png



4、结果分析

本题利用“Canny + 霍夫变换”相结合的方式，成功识别不同大小的硬币和蓝色线条。

使用 HoughLinesP 提取图像中所有直线段；对提取结果通过角度与位置进行聚类，合并断裂线段；最终以红色线段形式在图像上标注出蓝色线条的位置。

正确答案：

答案解析：

形态学 20250417

题量: 1 满分: 100

智能分析

100分

作答时间: 2025-04-25 11:39 至 2025-05-09 12:30

一. 简答题 (共1题, 100分)

1. (简答题)

(Morphologh) Please detect all letter "I" in the image

(include 1. Algorithm principle; 2. Key code;3. The results image; 4. Result analysis)



document.zip

34.77 KB

我的答案:

100 分

1、算法原理

- (1) 二值化: 将灰度图变为黑白图 (0/1), 黑字母变为前景。
- (2) 结构元素提取: 截取相对清晰的一个字母 "I" 的图块 (位于[405:425, 34:39]), 作为模板。
- (3) **腐蚀 (Erode)**: 对图像用 "I" 模板腐蚀, 保留完全匹配 "I" 形状的区域; 同时对背景图 (取反) 用反模板腐蚀, 排除不匹配区域。
- (4) **命中-未命中变换 (Hit-or-Miss)**: 将两腐蚀结果做与运算, 得到所有 "I" 的位置。【详见下方二图】
- (5) **膨胀 (Dilate)**: 用模板对命中区域膨胀, 恢复出完整的 "I" 图形。

The Hit-or-Miss Transformation

The morphological hit-or-miss transform is a basic tool for shape detection.

If B denotes the set composed of X and its back-ground, the match (or set of matches) of B in A , denoted as $A \odot B$, is

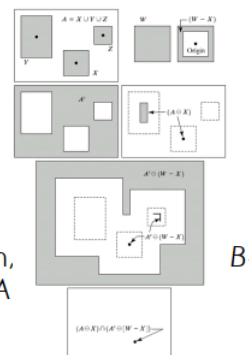
$$A \odot B = (A \ominus X) \cap [A^c \ominus (W - X)]$$

Letting $B = (B_1, B_2)$, where $B_1 = X$, $B_2 = (W - X)$,

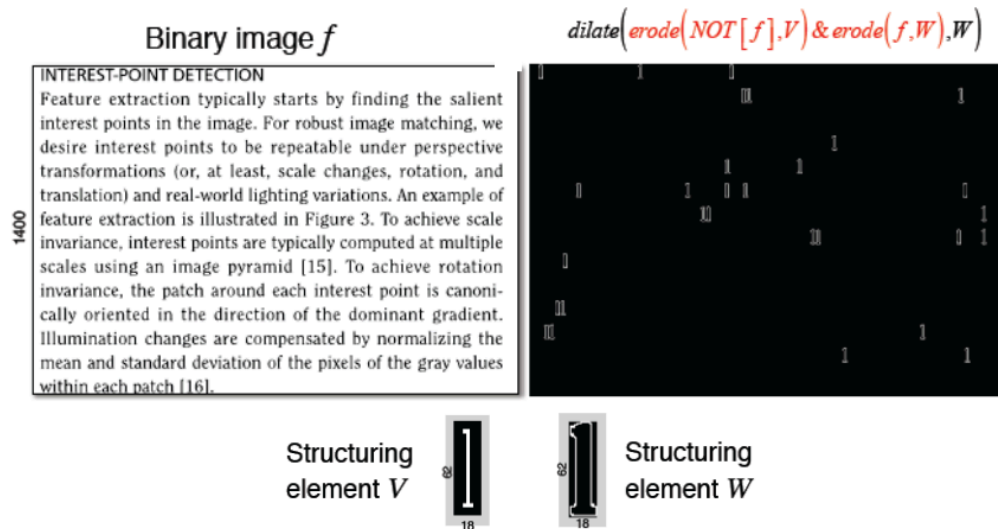
$$A \odot B = (A \ominus B_1) \cap (A^c \ominus B_2)$$

Thus, set $A \odot B$ contains all the (origin) points at which, finds a match ("hit") in A and B_2 found a match in A

This is called the *morphological hit-or-miss transform*.

 B_1

Recognition by Hit-or-Miss



2、关键代码

```
gray_img = cv2.imread('document.bmp', cv2.IMREAD_GRAYSCALE)

# 先进行二值化
_, binary_mask = cv2.threshold(gray_img, 100, 1, cv2.THRESH_BINARY_INV)
binary_mask = (binary_mask * 255).astype('uint8')

# 提取字母“l”（在倒数第三个找到一个相对清晰的）
letter_l = binary_mask[405:425, 34:39]
letter_l_inv = cv2.bitwise_not(letter_l)

# 命中-未命中变换（Hit-or-Miss）
erosion_fg = cv2.erode(binary_mask, letter_l)
erosion_bg = cv2.erode(cv2.bitwise_not(binary_mask), letter_l_inv)
hitmiss_result = cv2.bitwise_and(erosion_fg, erosion_bg)

# 使用“l”结构元素进行膨胀
letter_l_flip = cv2.flip(letter_l, -1)
dilated_result = cv2.dilate(hitmiss_result, letter_l_flip, anchor=(0, 0))

plt.figure(figsize=(10, 5))

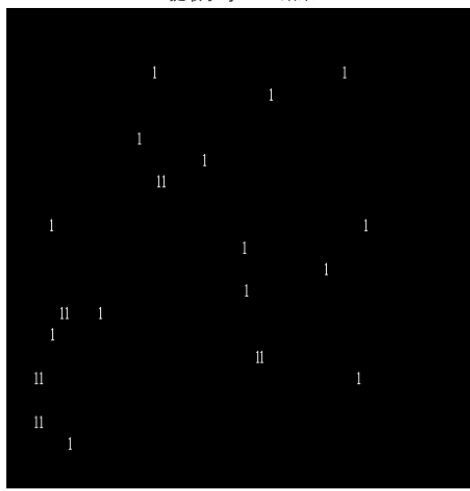
plt.subplot(1, 2, 1)
plt.title('二值图像（反转）')
plt.imshow(binary_mask, cmap='gray')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.title('提取字母“l”结果')
plt.imshow(dilated_result, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

3、结果图

current position of gastroscope in the stomach are shown in the gastroscope video to guide the endoscopist to reach biopsy positions. IGP systems, which are designed to augment and complement the surgeon's spatial understanding of the anatomy, have been widely used in neurosurgery and orthopedics where rigid anatomy assumptions apply [8]. To our knowledge, we are the first to introduce an IGP system in gastroscopy. Although the use of IGP in soft organ surgery is still in its infancy, research on providing intra-operative support in soft organ surgery has been developed rapidly [9-12]. The quantitative evaluation of accuracy, which reflects the validity range of the rigidity assumption, is very important for these applications. In our system, we chose a rigid model registration based on the carefully selected three pairs of correspondence points that should meet two conditions: (1) they must be easy for endoscopists to find so that the following examination time will be minimized; and, (2) their position should be more fixed, whereby the registration error caused by deformation will be minimized. The positioning accuracy of the system was validated via both phantom experiments and in vivo experiments.



4、结果分析

该算法能有效检测出原始图像document.bmp中形状匹配的“l”字符，适合小字符提取。

命中-未命中变换（Hit-or-Miss） 具备较强的模板匹配能力，适合结构固定的字符识别。

但是——也存在局限性：

对字符的旋转、缩放、畸变不具鲁棒性。

必须**手动选取**结构元素（需要按着精准的像素点去找），且对模板形状**依赖强**。

正确答案：

灰度形态学20250424

题量: 1 满分: 100

智能分析

100分

作答时间: 2025-04-25 11:39 至 2025-04-25 12:30

一. 简答题（共1题，100分）

1. (简答题)

(Morphology) Please process the following image with opening and closing operation respectively, then analyze

the results. (include 1. Algorithm principle; 2. Key code;3. The results image; 4. Result analysis)



blb1.zip
31.24 KB

我的答案:

100 分

1、算法原理

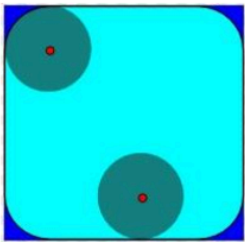
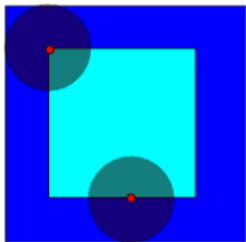
开操作

开运算通常会平滑物体的轮廓，打破狭窄的地峡，消除细长的突起。

集合A通过结构元素B的开运算定义为

$$A \circ B = (A \ominus B) \oplus B,$$

因此，A通过B的开运算是A通过B的腐蚀，随后是结果通过B的膨胀。



通过圆盘对深蓝色方块的开运算，得到带圆角的浅蓝色方块。

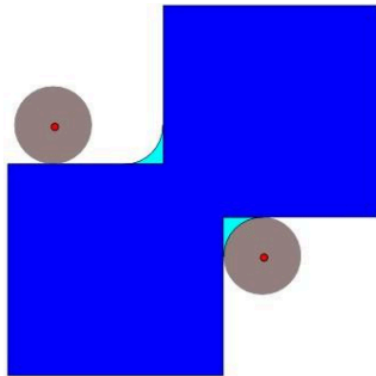
闭操作

闭运算也倾向于平滑轮廓部分，融合狭窄的裂缝和长薄的沟壑，消除小孔并填补间隙。

集合A通过结构元素B的闭运算定义为

$$A \bullet B = (A \oplus B) \ominus B,$$

通过 B 对 A 进行闭运算是把 A 进行膨胀 B，然后对结果进行腐蚀



通过圆盘对深蓝色形状（两个正方形的并集）进行闭合，得到深蓝色形状和浅蓝色区域的并集。

2、关键代码

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (10,10))
```

```
img_open = cv2.morphologyEx(img, cv2.MORPH_OPEN, kernel)
```

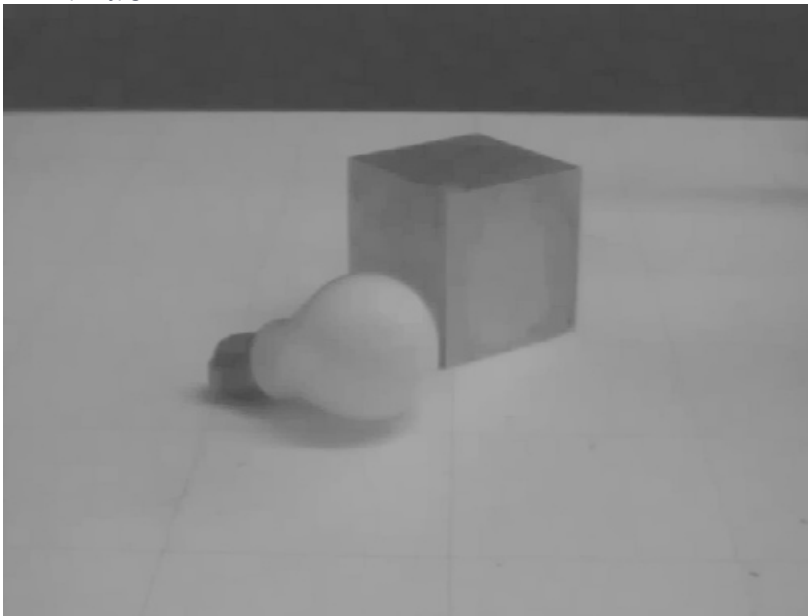
```
img_close = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
```

```
cv2.imwrite('blb1_open.jpg', img_open)
```

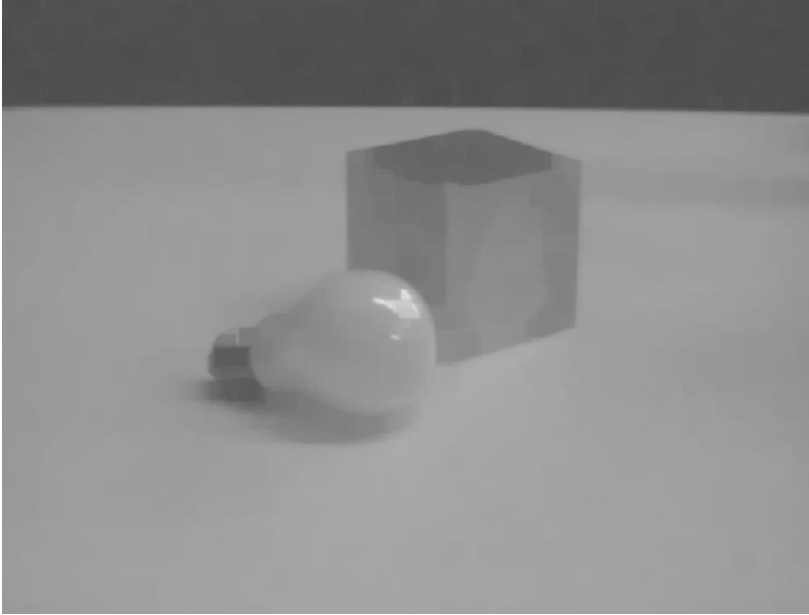
```
cv2.imwrite('blb1_close.jpg', img_close)
```

3、结果图

blb1_open.jpg



blb1_close.jpg



4、结果分析

(1) 开运算：灯泡上的光泽减弱（灯泡的亮部在腐蚀的时候几乎就已经消失），而正方体棱角更明显，图片局部变得黯淡。

这是因为开运算中先腐蚀（会抹去一些较亮、较凸出的小区域），再膨胀，在一定程度上会去除一些细小高亮斑点或噪声。

(2) 闭运算：灯泡上的光泽增强，小的暗斑或孔洞被补平，在图中显得更完整、更圆润。而正方体的棱角被模糊。桌面上的线条也被模糊，几乎消失不见。

这是因为闭运算先膨胀后腐蚀，可填补小凹陷或暗区，但同样可能在物体边界之外带来些许额外的亮区，导致物体或亮部区域在“闭运算”后变得稍微“大”一点。

另外两种操作都使用了较大（**kernal_size=(10,10)**）的矩形结构元素（**cv2.MORPH_RECT**），所以对原图中物体细节和局部亮度的影响都较为明显。

正确答案：

Geometric transformation

题量: 1 满分: 100

智能分析

100分

作答时间: 2025-05-09 11:44 至 2025-05-16 11:44

一. 简答题（共1题，100分）

1. (简答题)

(Interpolation) Please use the nearest neighbor interpolation method and bilinear interpolation method to enlarge the small below, and compare the magnification effect of the two methods.(Please write your own algorithm code, do not call the existing interpolation function)



interpolation.rar
68.35 KB

我的答案:

100 分

1、算法原理

最近邻插值:

Nearest-neighbor interpolation

■ E.g An image with dimension (w1 = 2, h1 = 2) is to be enlarged to (w2 = 4, h2 = 4).

$$\left. \begin{aligned} x_ratio &= \frac{w_1}{w_2} \\ y_ratio &= \frac{h_1}{h_2} \end{aligned} \right\} w_2, h_2 \neq 0$$
$$\begin{aligned} src_x &= dex_x \times x_ratio \\ src_y &= dex_y \times y_ratio \end{aligned}$$

	0	1
0	20	120
1	90	10

	0	1	2	3
0	20		120	
1				
2	90		10	
3				

Geometric transformation for mapping pixels.

$(0, 0) \times \frac{1}{2} \rightarrow (0, 0)$

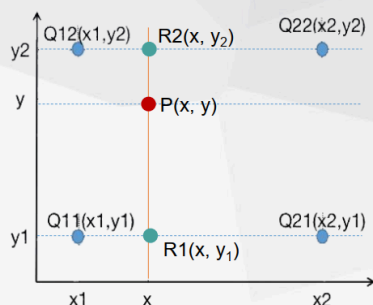
$(0, 2) \times \frac{1}{2} \rightarrow (0, 1)$

$(2, 0) \times \frac{1}{2} \rightarrow (1, 0)$

$(2, 2) \times \frac{1}{2} \rightarrow (1, 1)$

双线性插值:

Bilinear interpolation



Linear interpolation in x direction

$$f(R_1) = \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21})$$

$$f(R_2) = \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

Linear interpolation in y direction

$$f(P) = \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2)$$

$$f(x, y) = \frac{f(Q_{11})}{(x_2 - x_1)(y_2 - y_1)} (x_2 - x)(y_2 - y) + \frac{f(Q_{21})}{(x_2 - x_1)(y_2 - y_1)} (x - x_1)(y_2 - y) \\ + \frac{f(Q_{12})}{(x_2 - x_1)(y_2 - y_1)} (x_2 - x)(y - y_1) + \frac{f(Q_{22})}{(x_2 - x_1)(y_2 - y_1)} (x - x_1)(y - y_1)$$

$$f(x, y) = \frac{f(Q_{11})}{(x_2 - x_1)(y_2 - y_1)} (x_2 - x)(y_2 - y) + \frac{f(Q_{21})}{(x_2 - x_1)(y_2 - y_1)} (x - x_1)(y_2 - y) \\ + \frac{f(Q_{12})}{(x_2 - x_1)(y_2 - y_1)} (x_2 - x)(y - y_1) + \frac{f(Q_{22})}{(x_2 - x_1)(y_2 - y_1)} (x - x_1)(y - y_1)$$

$$\begin{aligned} x_2 &= x_1 + 1 \\ y_2 &= y_1 + 1 \end{aligned}$$

$$\begin{aligned} x_2 - x_1 &= 1 \\ y_2 - y_1 &= 1 \end{aligned}$$

$$f(x, y) = f(Q_{11})(x_2 - x)(y_2 - y) + f(Q_{21})(x - x_1)(y_2 - y) + f(Q_{12})(x_2 - x)(y - y_1) + f(Q_{22})(x - x_1)(y - y_1)$$

$$f(x, y) = f(Q_{11})w_{11} + f(Q_{21})w_{21} + f(Q_{12})w_{12} + f(Q_{22})w_{22}$$

2、关键代码

最近邻插值：

```
img = cv2.imread('Snoopy-small.jpg')
```

```
h,w,c = img.shape
```

```
new_h,new_w = h*10, w*10
```

```
res_nearest = np.zeros((new_h, new_w, c), dtype=img.dtype)
```

```
for i in range(new_h):
```

```
    for j in range(new_w):
```

```
        res_nearest[i,j] = img[i//10, j//10]
```

双线性插值：

```
img = cv2.imread('Snoopy-small.jpg')
```

```
h,w,c = img.shape
```

```
new_h,new_w = h*10, w*10
```

```
res_linear = np.zeros((new_h, new_w, c), dtype=np.uint8)
```

```
for i in range(new_h):
```

```
    for j in range(new_w):
```

```
        # 映射回原图坐标（浮点位置）
```

```
        x = i / 10
```

```
        y = j / 10
```

```
        x1 = int(np.floor(x))
```

```
        y1 = int(np.floor(y))
```

```
        x2 = min(x1 + 1, h - 1)
```

```
        y2 = min(y1 + 1, w - 1)
```

```
        dx = x - x1
```

```
        dy = y - y1
```

```
        for ch in range(c):
```

```
            Q11 = img[x1,y1,ch]
```

```
            Q21 = img[x2,y1,ch]
```

```
            Q12 = img[x1,y2,ch]
```

```
            Q22 = img[x2,y2,ch]
```

```
value = Q11 * (1 - dx) * (1 - dy) + Q21 * dx * (1 - dy) + Q12 * (1 - dx) * dy + Q22 * dx * dy  
res_linear[i,j,ch] = int(round(value))
```

3、结果图

nearest.jpg



linear.jpg



4、结果分析

最近邻插值：

放大后像素取其最近的原始像素值，不进行加权或平均。

运算速度快（计算简单）；图像有明显的马赛克；边缘锯齿明显

双线性插值：

对每个新像素，根据其周围最近4个原始像素进行加权平均，使用二维线性插值计算。

比最近邻更平滑；边缘柔和，但可能会造成轻微模糊；

特别是高倍放大时细节可能略模糊，本题是10倍，存在一些模糊。

正确答案：

Feature matching

题量: 2 满分: 100

智能分析

100分

作答时间: 2025-05-16 11:59 至 2025-06-06 11:59

一. 简答题 (共2题, 100分)

1. (简答题)

(Matching) Please use the HOG feature descriptor to detect the template image in the larger image.

cats.zip
70.74 KB

我的答案:

50 分

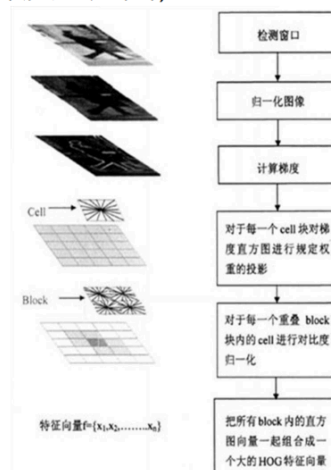
1、算法原理

HOG (Histogram of Oriented Gradient, 方向梯度直方图)

HOG特征提取实现过程

HOG特征提取方法就是将一个图片（要检测的目标或者扫描窗口）：

- 1) 灰度化（将图像看做一个x, y, z（灰度）的三维图像），归一化；
- 2) 计算图像每个像素的梯度（包括大小和方向）；
- 3) 将图像划分成小cells（例如6x6像素/cell）；
- 4) 统计每个cell的梯度直方图（不同梯度的个数），即可形成每个cell的描述特征；
- 5) 将每几个cell组成一个block（例如3x3个cell/block），一个block内所有cell的特征串联起来便得到该block的HOG特征。
- 6) 将图像内的所有block的HOG特征串联起来就可以得到该image（要检测的目标）的HOG特征了。这个就是最终的可供分类使用的特征向量了。



HOG (Histogram of Oriented Gradient, 方向梯度直方图)

■ 计算图像梯度

- 计算图像横坐标和纵坐标方向的梯度，并据此计算每个像素位置的梯度方向值；求导操作不仅能够捕获轮廓，人影和一些纹理信息，还能进一步弱化光照的影响。
- 图像中像素点 (x, y) 的梯度为：

$$G_x(x, y) = H(x + 1, y) - H(x - 1, y)$$

$$G_y(x, y) = H(x, y + 1) - H(x, y - 1)$$

- 其中 $G_x(x, y)$ 和 $G_y(x, y)$ 分别表示输入图像中像素点 (x, y) 的水平方向梯度和垂直方向梯度。 $H(x, y)$ 表示像素值。再以此计算出梯度的大小和方向：

-1	0	1
----	---	---

-1
0
1

$$G(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}$$

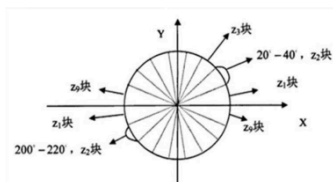
$$\alpha(x, y) = \tan^{-1} \frac{G_x(x, y)}{G_y(x, y)}$$

<https://learnopencv.com/histogram-of-oriented-gradients/>

HOG (Histogram of Oriented Gradient, 方向梯度直方图)

■ 构建梯度直方图

- 构建梯度直方图的目的是为局部图像区域提供一个编码，同时能够保持对图像中对象的姿势和外观的弱敏感性。
- 将图像分成若干个“单元格cell”（例如8x8像素/cell）。假设我们采用9个bin的直方图来统计这8x8个像素的梯度信息。也就是将cell的梯度方向360度分成9个方向块，如图所示：如果这个像素的梯度方向是20~40度，直方图第2个bin的计数就加对应梯度大小的权值，这样，对cell内每个像素映射到固定的角度范围，就可以得到这个cell的梯度方向直方图了，就是该cell对应的9维特征向量（因为有9个bin）。



2、关键代码

```
def compute_gradient(img):
```

```
    Gx = np.zeros_like(img, dtype=np.float32)
```

```
    Gy = np.zeros_like(img, dtype=np.float32)
```

```
    Gx[:, 1:-1] = img[:, 2:] - img[:, :-2]
```

```
    Gy[1:-1, :] = img[2:, :] - img[:-2, :]
```

```
    magnitude = np.sqrt(Gx**2 + Gy**2)
```

```
    return Gx, Gy, magnitude
```

```
# 模板梯度
```

```
gx_tpl, gy_tpl, mag_tpl = compute_gradient(template_gray)
```

```
# 模板大小
```

```
h, w = template_gray.shape
```

```
# ===== 滑动窗口匹配 =====
```

```
max_score = -np.inf
```

```
best_x, best_y = 0, 0
```

```
# 遍历原图中的所有可能位置（逐像素滑动）
```

```
for y in range(0, img_gray.shape[0] - h):
```

```
    for x in range(0, img_gray.shape[1] - w):
```

```
        patch = img_gray[y:y+h, x:x+w]
```

```
        gx_p, gy_p, mag_p = compute_gradient(patch)
```

```
# 使用方向余弦相似度作为匹配指标（越大越相似）
```

```
score = np.sum(np.cos(gx_tpl - gx_p) * mag_tpl * mag_p)
```



```

if score > max_score:
    max_score = score
    best_x, best_y = x, y
    best_gx, best_gy, best_mag = gx_p, gy_p, mag_p

```

```

matched_img = img_color.copy()
cv2.rectangle(matched_img, (best_x, best_y), (best_x + w, best_y + h), (0, 255, 0), 2)

```

```

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
fig.suptitle("HOG模板匹配流程图", fontsize=18)

```

```

# 输出上排：原图 / 模板 / 匹配结果（略）
# 输出下排：水平梯度 / 垂直梯度 / 梯度幅值（略）

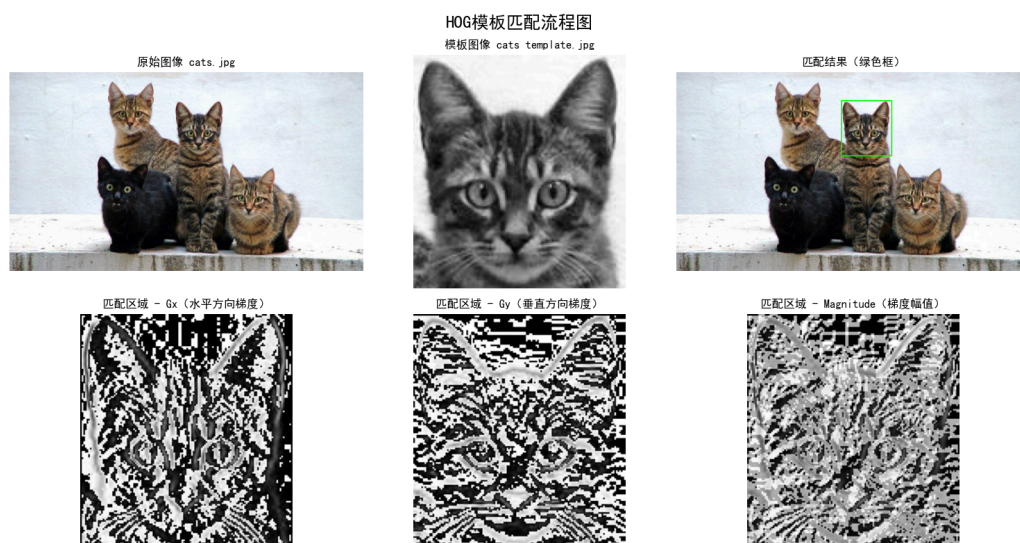
```

```

plt.tight_layout()
plt.show()

```

3、结果图



4、结果分析

在匹配结果（图3）中，绿色框准确地框出了与模板图像（图2）的猫脸最相似的区域。

滑动窗口正确覆盖了模板相似区域，梯度相似度作为匹配指标有效区分了目标猫脸。

而下方三图呈现的是匹配到的区域在梯度域的响应情况：

Gx（水平方向梯度）：边缘从左到右的强度变化——耳朵、眼睛边缘较亮，反映强边界结构；

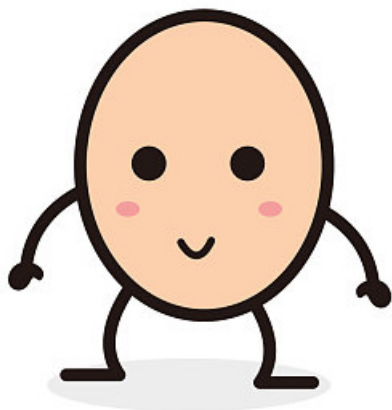
Gy（垂直方向梯度）：边缘从上到下的强度变化——鼻子、眼睛下方等区域呈现明显亮度跳变；

Magnitude（梯度幅值）：综合强度信息——保留整体形状轮廓，与模板基本吻合。

正确答案：

2. (简答题)

(Matching) Please use the SIFT descriptor to detect the template image in the larger image.





我的答案:

1、算法原理

SIFT (Scale-invariant feature transform) 描述子

■ 在SIFT 算法的步骤

- (1) 构造高斯差分尺度空间
- (2) 检测尺度空间极值点
- (3) 精确定位极值点，去除不好的特征点
- (4) 给特征点赋值一个128维的方向参数。每个关键点都包含三个信息：位置、尺度和方向。
- (5) 关键点描述子的生成

SIFT (Scale-invariant feature transform) 描述子

■ 尺度空间的生成

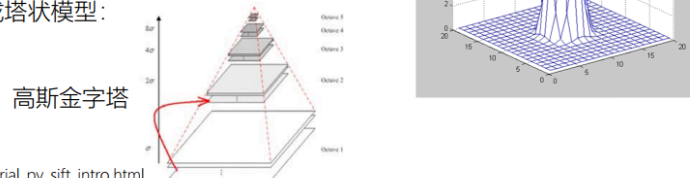
尺度空间的目的是模拟图像数据的多尺度特性，一幅二维图形的尺度空间定义为：

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

其中 $G(x, y, \sigma)$ 是尺度可变高斯函数 $G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2}$, (x, y) 是空间坐标, σ 是尺度坐标, 决定图像的平滑程度, 大的 σ 值对应大尺度低分辨率。

■ 高斯金字塔的构建

在对图像做不同尺度的高斯模糊后，再对图像做降采样并由大到小，从下到上构成塔状模型：



https://docs.opencv.org/3.4/da/df5/tutorial_py_sift_intro.html

SIFT (Scale-invariant feature transform) 描述子

■ 高斯差分金字塔

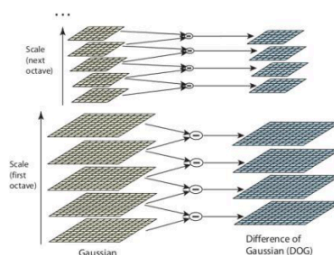
构建尺度空间的目的是为了检测出在不同的尺度下都存在的特征点，而检测特征点较好的算

子是 $\Delta^2 G$ (高斯拉普拉斯, LoG), 其中 $\Delta^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ 。为了减少计算量, 通常使用DoG

(差分高斯) 来近似计算, 设 k 为相邻两个高斯尺度空间的比例因子, 则高斯差分尺度空间定义如下:

$$\begin{aligned} D(x, y, \sigma) &= [G(x, y, k\sigma) - G(x, y, \sigma)] * I(x, y) \\ &= L(x, y, k\sigma) - L(x, y, \sigma) \end{aligned}$$

其中 $L(x, y, \sigma)$ 是图像的高斯尺度空间



https://docs.opencv.org/3.4/da/df5/tutorial_py_sift_intro.html

2、关键代码

```
# 创建SIFT特征检测器
```

```
sift = cv2.SIFT_create()
```

```
# 创建多尺度图像金字塔
```

```
def create_pyramid(image, num_scales=4, scale_factor=1.2):
```

```
    pyramid = [image]
```

```
    for i in range(1, num_scales):
```

```

scaled_image = cv2.resize(image, None, fx=1.0 / scale_factor ** i, fy=1.0 / scale_factor ** i,
                           interpolation=cv2.INTER_AREA)
pyramid.append(scaled_image)
return pyramid

template_pyramid = create_pyramid(template)
image_pyramid = create_pyramid(image)

# 存储所有匹配结果
all_matches = []
all_keypoints_template = []
all_keypoints_image = []
all_descriptors_template = []
all_descriptors_image = []

for i, scaled_template in enumerate(template_pyramid):
    # 检测模板图像和待检测图像中的关键点和描述符
    keypoints_template, descriptors_template = sift.detectAndCompute(scaled_template, None)
    keypoints_image, descriptors_image = sift.detectAndCompute(image_pyramid[i], None)

    # 检查是否检测到关键点
    if keypoints_template is None or keypoints_image is None:
        continue

    # 使用FLANN匹配器进行特征匹配
    FLANN_INDEX_KDTREE = 1
    index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)

    # 检查描述符是否存在
    if descriptors_template is None or descriptors_image is None:
        continue

    # 确保描述符有足够的数量进行匹配
    if len(descriptors_template) < 2 or len(descriptors_image) < 2:
        continue

    matches = flann.knnMatch(descriptors_template, descriptors_image, k=2)

    # 使用Lowe的比率测试筛选出好的匹配
    good_matches = []
    for m, n in matches:
        if m.distance < 0.7 * n.distance:
            good_matches.append(m)

    all_matches.append(good_matches)
    all_keypoints_template.append(keypoints_template)
    all_keypoints_image.append(keypoints_image)
    all_descriptors_template.append(descriptors_template)
    all_descriptors_image.append(descriptors_image)

# 在原始图像上绘制所有匹配结果
image_with_all_boxes = image.copy()
for i, good_matches in enumerate(all_matches):
    if len(good_matches) > 4:
        # 获取当前尺度的关键点
        keypoints_template = all_keypoints_template[i]
        keypoints_image = all_keypoints_image[i]

        # 检查关键点数量是否足够
        if len(keypoints_template) == 0 or len(keypoints_image) == 0:

```

continue

```
src_pts = np.float32([keypoints_template[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
dst_pts = np.float32([keypoints_image[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
```

```
# 调整尺度
```

```
scale_factor = 1.2 ** i
```

```
src_pts *= scale_factor
```

```
dst_pts *= scale_factor
```

```
# 计算单应性矩阵
```

```
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
```

```
if M is not None:
```

```
    # 计算模板图像的边界在图像上的位置
```

```
    h, w = template.shape
```

```
    pts = np.float32([[0, 0], [0, h - 1], [w - 1, h - 1], [w - 1, 0]]).reshape(-1, 1, 2)
```

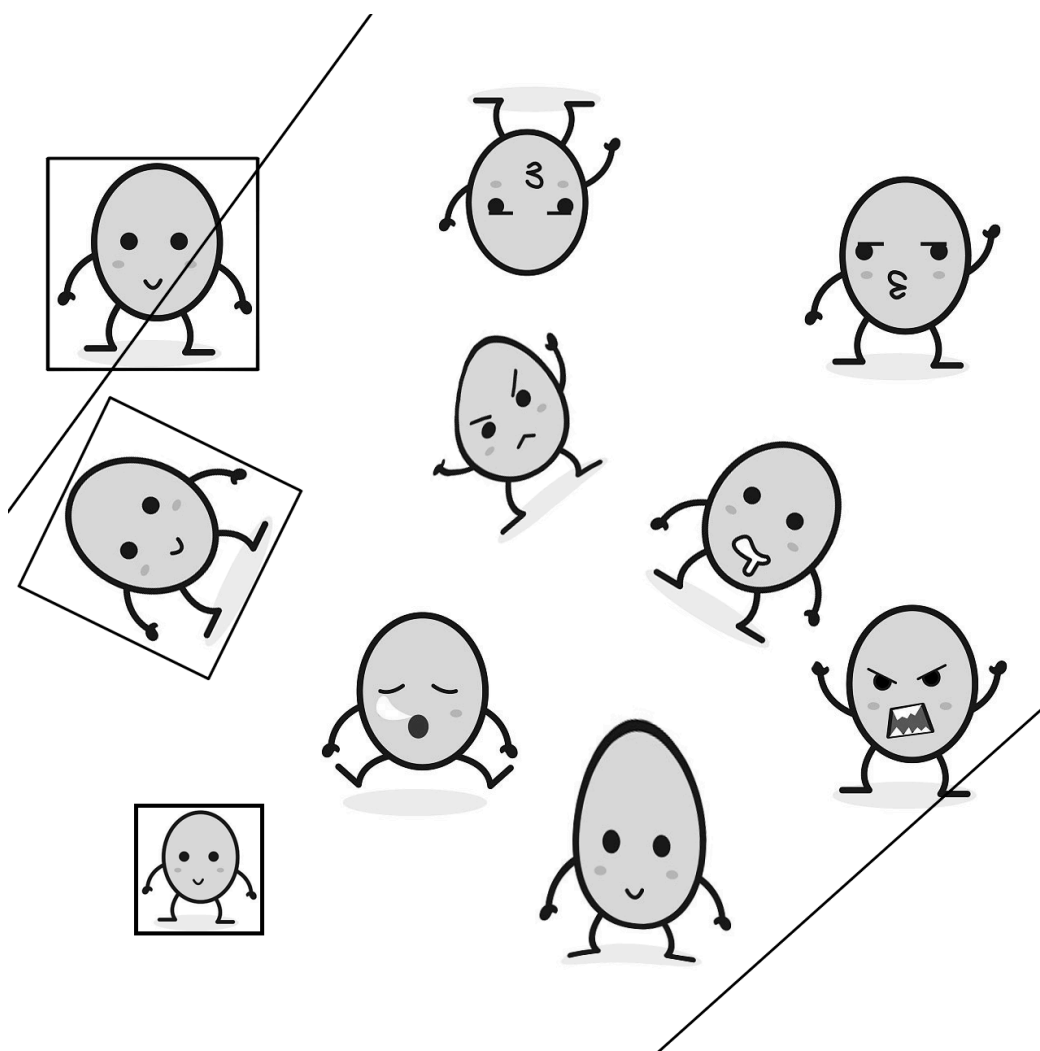
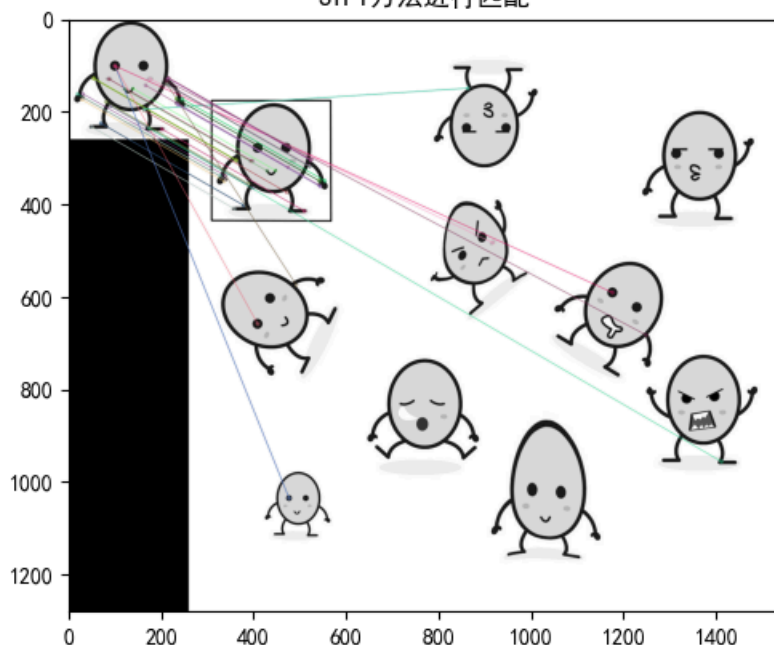
```
    dst = cv2.perspectiveTransform(pts, M)
```

```
# 在图像上绘制边框
```

```
image_with_all_boxes = cv2.polylines(image_with_all_boxes, [np.int32(dst)], True, (0, 0, 255), 2,
                                     cv2.LINE_AA)
```

3、结果图（含匹配过程和框选结果）

SIFT方法进行匹配



4、结果分析

模板图像与目标图像（大图）中3个鸡蛋进行了特征匹配，包括不同位置、大小和方向的鸡蛋。

对于不同大小的鸡蛋：通过**多尺度图像金字塔**的使用，代码成功识别了不同大小的鸡蛋。图中可以看到，左下角的较小鸡蛋也被框选出来。这表明多尺度处理在识别不同尺寸的模板方面是有效的。

匹配点分布：从连接线可以看出，特征匹配点主要集中在鸡蛋的**轮廓和关键特征区域**，如面部表情和四肢位置，这表明特征检测器能够捕捉到这些显著特征。

匹配精度：大部分匹配点能够准确对应到鸡蛋的关键区域，说明**特征匹配算法（本题为SIFT）**在该场景下具有较高的精度。

旋转鸡蛋的识别：图中可以看到，旋转后的鸡蛋也被成功框选，这表明算法对旋转具有一定的不变性。特征描述符能够捕捉到旋转后的特征，使得匹配得以成功。

鲁棒性：特征匹配过程对旋转的鲁棒性较好，即使鸡蛋发生一定程度的旋转，仍然能够识别并框选。

实验一：基于视觉方法的工件精细尺寸测量

一、实验目的

1. 熟悉机器视觉算法中的常用图像滤波、阈值分割、形态学算法；
2. 熟悉机器视觉算法中常用的目标定位算法；
3. 熟悉机器视觉算法中常用的边缘检测、直线拟合算法；
4. 能够综合运用所学知识解决一些简单的实际问题。

二、实验内容

1、目标定位

- 1.1 使用阈值分割算法将目标工件从图像中分离出来，并给出工件在图像中的准确位置。

采用 **OTSU 自适应阈值分割算法**对灰度图像进行处理，通过计算像素直方图的双峰特性自动确定最优阈值，将工件从背景中分离出来。由于工件区域在二值化后呈现为黑色，通过**像素值取反操作**（255-binary）使工件变为白色区域，便于后续轮廓提取。随后利用 5×5 矩形核进行形态学闭运算，通过**先膨胀后腐蚀**的操作填补工件内部的细小空洞，并连接邻近的边缘，提升轮廓的完整性。最后通过**查找图像外轮廓**（RETR_EXTERNAL）并**过滤面积小于 1000 像素的噪声轮廓**，实现对目标工件的精准分割，同时按外接矩形的 y 坐标排序，确定工件在图像中的上下位置顺序。

- 1.2 结合工件的整体尺寸信息，对线扫相机图像进行自适应切割，还原完整工件。

基于工件的整体尺寸信息，以每个工件的中心点为基准，向四周扩展固定像素（宽 2200 像素，高 800 像素）进行 **ROI 区域切割**。为避免边缘信息丢失，切割时在边界添加 10 像素的 Padding，确保完整还原线扫相机图像中的工件形态。切割后的 ROI 区域通过绿色矩形框在原始图像中可视化标注，同时标记工件编号与中心点位置，实现对工件的精准定位。

2、边缘检测&直线拟合。

- 2.1 结合预先给定的检测目标信息，准确地提取目标边缘。

针对检测目标的边缘特性，首先采用 **CLAHE（自适应直方图均衡化）**对 ROI 区域进行预处理，通过设置 clipLimit=2.0 和 tileGridSize=(4,4)，增强低对比度工件的边缘细节。随后利用 5×5 高斯核进行模糊去噪，减少图像噪声对边缘检测的干扰。边缘提取阶段采用 Sobel 算子分别计算 x 和 y 方向的梯度幅值，通过加权融合（权重各 0.5）得到综合梯度图，再通过阈值 15 的二值化处理，最终获得清晰的目标边缘轮廓。

- 2.2 通过对目标边缘点进行拟合，得到目标的直线方程。

对提取的边缘点采用**霍夫变换（HoughLinesP）**进行直线检测，设置距离精度 rho=1 像素、角度精度 theta= $\pi/180$ 弧度，通过累加器阈值 threshold=50 过滤无效直线，并利用 minLineLength=200 和 maxLineGap=10 筛选并合并邻近线段。对于检测到的水平直线，按 y 坐标排序后，将 y 坐标差小于 10 像素的线段合并，最终通过合并后线段的中点坐标拟合得到目标直线方程，例如水平直线方程表示为 y=640（对应工件 1 的上边缘）。

3、目标尺寸检测

3.1 结合检测目标的直线方程，计算各个检测尺寸的值。

结合检测得到的直线方程，提取目标序号对（如 442-443、445-448）的 y 坐标值，计算其像素距离。根据实验参数，垂直方向单位像素尺寸为 0.025mm/像素，将像素距离转换为实际物理距离。例如，序号对 445-448 的像素距离为 180 像素，则实际距离为 $180 \times 0.025 = 4.50\text{mm}$ 。通过该方法依次计算各检测目标的尺寸值，并与标准值对比，验证测量精度。

三、实验中的参数

1、单位像素尺寸

水平方向：0.027mm/像素 垂直方向：0.025mm/像素

2、检测目标



三、实验结果

3.1 工件中心点标注后的图片

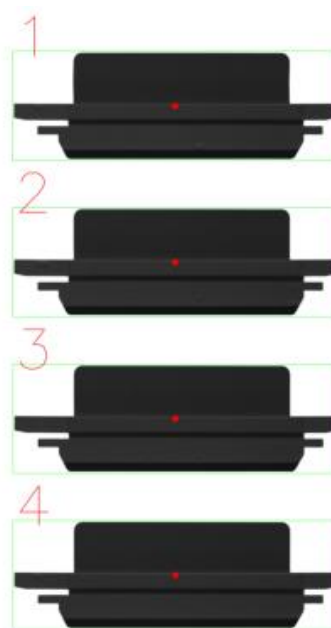


图 1 四个工件的中心点位置标注

3.2 测量参数表格

工件编号	中心点坐标	检测对象编号 1	检测对象编号 2	测量值 (mm)	标准值 (mm)
1	(2301, 629)	442	443	1.43	1.50
		445	448	3.53	-
		444	445	8.15	-
		447	448	0.95	-
2	(2302, 1630)	442	443	1.40	1.50
		445	448	3.50	-
		444	445	8.08	-
		447	448	0.93	-
3	(2304, 2634)	442	443	1.40	1.50
		445	448	3.40	-
		444	445	8.20	-
		447	448	0.93	-
4	(2304, 3636)	442	443	1.45	1.50
		445	448	3.45	-
		444	445	8.10	-
		447	448	0.85	-

四、总结与体会

1.请记录调试过程中，碰到的问题（至少 3 个），以及相应的解决方法。

（1）阈值分割效果不佳：直接使用固定阈值分割无法有效分离工件与背景。

最初尝试的是使用固定阈值 128 进行二值化，但原始图像对比度低，分割效果很差。意识到图像不同区域的最佳阈值可能不同，需要自适应方法。

查询资料，尝试了几种自适应阈值方法（ADAPTIVE_THRESH_MEAN_C 和 ADAPTIVE_THRESH_GAUSSIAN_C），效果仍不理想！最终尝试采用 OTSU 算法（THRESH_OTSU），该算法会自动计算最佳全局阈值，无需手动调参。

```
_ , binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

注：参数中的 0 被 OTSU 算法忽略，仅占位用；由于工件在原图中为深色，而后续处理需要白色前景，故添加 255 - binary 反转。

（2）多工件识别困难：工件密集排列，传统轮廓检测易将多个工件误判为一个。

最初使用 RETR_LIST 模式检测所有轮廓，但重叠区域会产生多个嵌套轮廓，难以区分。

意识到需要只检测外部轮廓，于是尝试 RETR_EXTERNAL 模式，仅保留最外层轮廓，成功解决重叠问题。需要调节面积阈值，先从 500 开始尝试，发现过小会保留噪声，最终设为 1000（根据实际图像尺寸调整）。

```
contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
min_area = 1000
```

(3) 工件边界提取不准确：工件边界框包含过多背景区域，影响后续分析精度。

原先直接使用 `cv2.boundingRect()` 获取的边界框紧贴工件，但某些特征可能位于边缘。需要添加适当 `padding`。

先从 5 像素 `padding` 开始，发现部分特征仍被截断；增加到 15 像素时，边界框过大引入过多背景。最终确定 10 像素为平衡点，并添加边界检查防止超出图像范围。

```
padding = 10
x1 = max(x - padding, 0) # 确保不超出左边界
y1 = max(y - padding, 0) # 确保不超出上边界
x2 = min(x + w + padding, gray.shape[1]) # 确保不超出右边界
y2 = min(y + h + padding, gray.shape[0]) # 确保不超出下边界
```

(4) 直线检测准确率低：Canny 边缘检测结果受噪声影响严重；霍夫变换检测到大量短线和伪直线，难以准确识别真实特征线。

Canny 边缘检测对噪声敏感，Sobel 算子计算梯度幅值可能更稳健。概率霍夫变换（`HoughLinesP`）比标准霍夫变换更适合检测线段。

对于 Sobel 算子，尝试不同核大小（3/5/7），发现 `ksize=5` 在边缘连续性和噪声抑制间平衡最佳；而霍夫变换则有三个参数需调：`threshold`：从 30 开始，逐渐增加到 50，减少假阳性；`minLineLength`：从 100 开始，增加到 200，过滤短线；`maxLineGap`：从 5 开始，增加到 10，允许连接小间隙。

```
sobelx = cv2.Sobel(gray_enhanced, cv2.CV_64F, 1, 0, ksize=5)
sobely = cv2.Sobel(gray_enhanced, cv2.CV_64F, 0, 1, ksize=5)
lines = cv2.HoughLinesP(edges, rho=1, theta=np.pi/180, threshold=50, minLineLength=200, maxLineGap=10)
```

(5) 水平直线合并困难：同一特征线被检测为多条相邻线段，导致测量结果不准确。

真实水平线的 y 坐标差异很小（通常 <5 像素），而假直线的 y 坐标差异较大。可以根据垂直距离合并线段。

合并阈值：从 5 像素开始，发现某些真实线段未被合并；增加到 15 像素时，开始合并非同一水平线。最终确定 **10 像素** 为最佳阈值，并添加 x 方向的扩展逻辑（取最小 x 和最大 x ）。

```
merge_threshold = 10 # 垂直距离阈值
if abs(y - last_y) < merge_threshold:
    new_y = int((last_y + y) / 2)
    new_x_start = min(last_x_start, x_start)
    new_x_end = max(last_x_end, x_end)
```

2. 其他

(1) 算法改进及关键调参决策

(i.) 边缘检测方法选择：最初尝试 Canny 边缘检测，参数 `threshold1=50` 和 `threshold2=150`，但结果过于敏感。改用 Sobel 算子后，发现通过加权组合 x 和 y 方向梯度（ $0.5*|sobelx|+0.5*|sobely|$ ）能更好地保留水平边缘。

(ii.) 霍夫变换参数优化：`rho=1`：保持高精度距离检测。`theta=np.pi/180`：角度精度设为 1 度，平衡计算量和精度。`threshold`：从 30 增加到 50，减少假阳性。`minLineLength`：从 100 增加到 200，过滤短线。

(iii.) 自适应直方图均衡化：尝试标准直方图均衡化（`cv2.equalizeHist()`），但过度增强噪声。改用 CLAHE（对比度受限自适应直方图均衡化），参数：`clipLimit=2.0`：限制对比度增强程度，防止噪声放大；`tileGridSize=(4,4)`：将图像分成 4×4 块，局部自适应增强。

(iv.) 直线方向判断：最初使用角度判断水平线（ $|\text{angle}| < 10^\circ$ ），但计算开销大。简化为比较 dx

和 dy ($dx > dy$ and $dy < 5$), 更高效且直观。

(2) 总结与经验

(i.) 调参策略: 从默认参数开始, 逐步调整关键参数; 每次只改变一个参数, 观察效果后再调整下一个; 记录每个参数的变化范围和最佳值, 形成参数调优日志。

(ii.) 算法选择原则: 优先使用简单算法, 仅在必要时增加复杂度; 针对特定问题选择专用算法 (如 Sobel 更适合水平边缘检测)。

(iii.) 鲁棒性设计: 添加边界检查和异常处理, 防止参数超出合理范围; 对关键参数设置默认值, 确保在参数不合理时仍能运行。

(iv.) 可视化辅助调参: 添加中间结果可视化, 直观评估每一步处理效果, 快速定位问题。

五、带有注释的程序源代码

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设黑体, 解决中文显示问题

# ----- 1. 预处理 -----
def preprocess():
    gray = cv2.imread('14raw.bmp', cv2.IMREAD_GRAYSCALE) # 转灰度
    original = cv2.imread('14raw.bmp') # 原图

    # 使用 OTSU 算法进行自适应阈值分割, 自动确定最佳阈值
    _, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    binary = 255 - binary # 黑白反转, 使前景工件为白色
    # 创建 5x5 矩形结构元素, 用于形态学操作
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    # 形态学闭操作: 先膨胀后腐蚀, 填充小孔洞并连接相邻区域
    binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

    return gray, original, binary

# ----- 2. 工件分割与可视化 -----
def segment_parts(gray, original, binary):
    """
    查找轮廓并分割出每个工件区域, 计算中心点并可视化在原图上
    参数: gray (灰度图)、original (原始彩色图)、binary (二值化图)
    返回: 存储每个工件灰度图的列表 parts
    """
    # 查找二值图像中的外部轮廓
    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    min_area = 1000 # 最小轮廓面积阈值, 过滤掉噪声和小区域
    valid_contours = []
    # 筛选有效工件轮廓
    for cnt in contours:
        area = cv2.contourArea(cnt)
```

```

        if area > min_area:
            valid_contours.append(cnt)

# 计算每个有效轮廓的边界矩形
boxes = [cv2.boundingRect(cnt) for cnt in valid_contours]
boxes.sort(key=lambda box: box[1]) # 按 y 坐标排序，确保从上到下排列工件

parts = []
# 处理每个工件区域
for i, (x, y, w, h) in enumerate(boxes):
    padding = 10 # 边界扩展像素数
    # 计算带 padding 的区域边界，确保不超出图像范围
    x1 = max(x - padding, 0)
    y1 = max(y - padding, 0)
    x2 = min(x + w + padding, gray.shape[1])
    y2 = min(y + h + padding, gray.shape[0])
    # 提取工件区域的灰度图像
    part_img = gray[y1:y2, x1:x2]
    parts.append(part_img)

    # 在原始彩色图上标记工件：绘制边界框、编号和中心点
    cv2.rectangle(original, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(original, f'{i + 1}', (x1 + 10, y1 + 30),
                 cv2.FONT_HERSHEY_SIMPLEX, 12, (0, 0, 255), 3, cv2.LINE_AA)
    center_x = x1 + (x2 - x1) // 2
    center_y = y1 + (y2 - y1) // 2
    cv2.circle(original, (center_x, center_y), 20, (0, 0, 255), -1)
    print(f'工件 {i+1} 的中心点坐标: ({center_x}, {center_y})')
print("——" * 20)

# 可视化工件在原图中的位置
plt.figure()
plt.imshow(cv2.cvtColor(original, cv2.COLOR_BGR2RGB))
plt.title("四个工件的中心点位置标注")
plt.axis('off')
plt.show()

return parts

# ----- 3. 单个工件处理函数（增强、边缘检测、霍夫变换、合并直线） -----
---
def process_single_part(part_gray, idx):
    """
    对单个工件灰度图进行处理：增强、边缘检测、霍夫变换检测直线、合并直线并可视化结果
    参数：part_gray（单个工件的灰度图）、part_idx（工件索引，从 0 开始）
    """

```

```

# 自适应直方图均衡化，增强图像对比度
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(4, 4))
gray_enhanced = clahe.apply(part_gray)
# 高斯模糊降噪，平滑图像
gray_enhanced = cv2.GaussianBlur(gray_enhanced, (5, 5), 0)

# 使用 Sobel 算子进行边缘检测
sobelx = cv2.Sobel(gray_enhanced, cv2.CV_64F, 1, 0, ksize=5) # x 方向梯度
sobely = cv2.Sobel(gray_enhanced, cv2.CV_64F, 0, 1, ksize=5) # y 方向梯度
# 计算梯度幅值，结合 x 和 y 方向梯度
sobel_magnitude = 0.5 * np.abs(sobelx) + 0.5 * np.abs(sobely)
# 归一化并转换为 8 位无符号整数
sobel_magnitude = np.uint8(255 * sobel_magnitude / np.max(sobel_magnitude))
# 阈值处理，提取显著边缘
_, edge_binary = cv2.threshold(sobel_magnitude, 15, 255, cv2.THRESH_BINARY)

edges = edge_binary.copy()

# 使用概率霍夫变换检测直线
lines = cv2.HoughLinesP(edges,
                          rho=1,          # 距离精度（像素）
                          theta=np.pi / 180, # 角度精度（弧度）
                          threshold=50,      # 最小投票数，阈值越高检测越严格
                          minLineLength=200, # 最小线段长度
                          maxLineGap=10)     # 线段连接最大间隙

# 提取并合并水平直线
splines = []
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        dx = abs(x2 - x1)
        dy = abs(y2 - y1)
        # 判断是否为近似水平直线（x 方向变化远大于 y 方向）
        if dx > dy and dy < 5:
            y_avg = int((y1 + y2) / 2)
            splines.append((y_avg, min(x1, x2), max(x1, x2)))
splines.sort(key=lambda l: l[0]) # 按 y 坐标排序

# 合并相近的水平直线
newlines = []
merge_threshold = 10 # 合并阈值（像素）
for y, x_start, x_end in splines:
    if not newlines:
        newlines.append([y, x_start, x_end])
    else:

```

```

        last_y, last_x_start, last_x_end = newlines[-1]
        # 如果两条线垂直距离小于阈值，则合并
        if abs(y - last_y) < merge_threshold:
            new_y = int((last_y + y) / 2)
            new_x_start = min(last_x_start, x_start)
            new_x_end = max(last_x_end, x_end)
            newlines[-1] = [new_y, new_x_start, new_x_end]
        else:
            newlines.append([y, x_start, x_end])

# 可视化检测结果
line_img = cv2.cvtColor(part_gray, cv2.COLOR_GRAY2BGR)
for y, x_start, x_end in newlines:
    cv2.line(line_img, (x_start, y), (x_end, y), (0, 0, 255), 2)

plt.figure(figsize=(12,6))
plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB))
plt.title(f'工件 {idx+1} 合并后的水平直线')
plt.axis('off')
plt.show()

# 计算各水平线方程（形如 y=?）
ys = [line[0] for line in newlines]
ys.sort()
# 将检测到的水平线 y 坐标映射到特定编号
keys = ["444", "445", "447", "448", "442", "443"]
ys_dic = dict(zip(keys, ys))
# 计算特定水平线之间的距离（像素单位）
d_tu = [ys_dic["443"]-ys_dic["442"],
        ys_dic["448"]-ys_dic["445"],
        ys_dic["445"]-ys_dic["444"],
        ys_dic["448"]-ys_dic["447"],]
# 将像素距离转换为实际物理距离（比例为 0.025）
d_shiji = [round(d*0.025,2) for d in d_tu]
print(f'工件 {idx+1} 的水平线数量（已合并）:', len(newlines))
print(f'工件 {idx+1} 水平线的 y 坐标:', ys)
print(f'工件 {idx+1} 相邻水平线的距离:',
      "\n442-443:",d_shiji[0],"\n445-448:",d_shiji[1],
      "\n444-445:",d_shiji[2],"\n447-448:",d_shiji[3])
print("——" * 20)

# ----- 4. 主程序流程 -----
if __name__ == "__main__":
    # 1. 图像读取与预处理
    gray, original, binary = preprocess()
    # 2. 工件分割与可视化

```

```
parts = segment_parts(gray, original, binary)
# 3. 循环处理每个工件
for idx, part in enumerate(parts):
    process_single_part(part, idx)
```

附程序输出结果:

工件 1 的中心点坐标: (2301, 629)
工件 2 的中心点坐标: (2302, 1630)
工件 3 的中心点坐标: (2304, 2634)
工件 4 的中心点坐标: (2304, 3636)

工件 1 的水平线数量 (已合并): 6
工件 1 水平线的 y 坐标: [14, 340, 443, 481, 635, 692]
工件 1 相邻水平线的距离:
442-443: 1.43
445-448: 3.53
444-445: 8.15
447-448: 0.95

工件 2 的水平线数量 (已合并): 6
工件 2 水平线的 y 坐标: [15, 338, 441, 478, 635, 691]
工件 2 相邻水平线的距离:
442-443: 1.4
445-448: 3.5
444-445: 8.08
447-448: 0.93

工件 3 的水平线数量 (已合并): 6
工件 3 水平线的 y 坐标: [14, 342, 441, 478, 634, 690]
工件 3 相邻水平线的距离:
442-443: 1.4
445-448: 3.4
444-445: 8.2
447-448: 0.93

工件 4 的水平线数量 (已合并): 6
工件 4 水平线的 y 坐标: [14, 338, 442, 476, 633, 691]
工件 4 相邻水平线的距离:
442-443: 1.45
445-448: 3.45
444-445: 8.1
447-448: 0.85

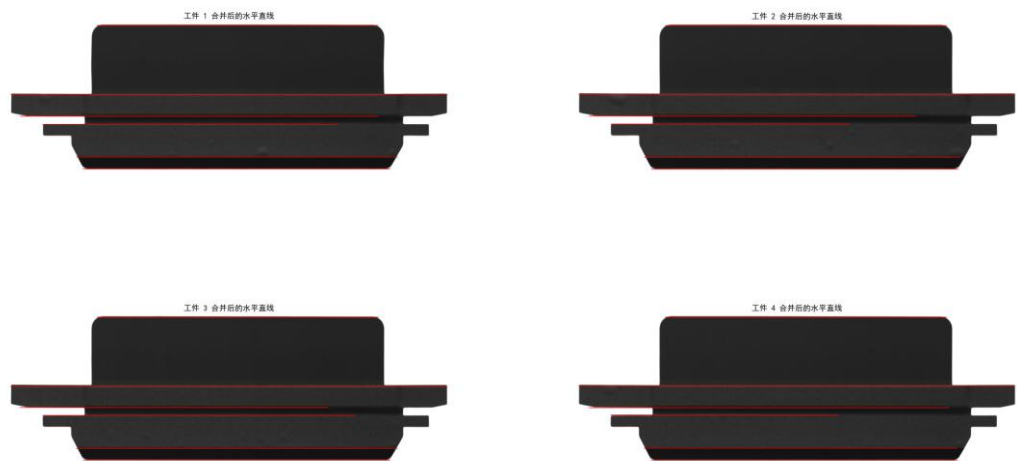


图 2 工件 1~4 合并后的水平直线可视化

实验二：基于视觉方法的珍珠圆度测量

一、实验目的

1. 熟悉机器视觉算法中的常用图像滤波、阈值分割、形态学等算法；
2. 熟悉机器视觉算法中常用的目标定位算法；
3. 熟悉机器视觉算法中常用的边缘检测、特征计算；
4. 能够综合运用所学知识解决一些简单的实际问题。

二、实验内容

1、定位各粒珍珠位置

1.1 图像预处理与轮廓提取

首先，利用 OpenCV 库对输入的珍珠图像进行处理。通过 `cv2.imread` 读取图像后，依次执行**灰度转换**（`cv2.cvtColor` 转 `COLOR_BGR2GRAY`）、**高斯模糊**（`cv2.GaussianBlur`， 5×5 核）消除噪声。为强化珍珠边缘，先经**形态学膨胀**（ 5×5 核，迭代 1 次）扩大珍珠区域，再用 **Canny 边缘检测** 突出轮廓，最后通过 `cv2.bitwise_and` 融合边缘与原图，为后续阈值分割做准备。

接着，采用**自适应阈值分割**（`cv2.adaptiveThreshold`，高斯加权、反二值化）将图像转化为黑白二值图，分离珍珠与背景。再通过**形态学开闭运算**（ 3×3 核，闭运算迭代 2 次、开运算迭代 1 次），填充珍珠内部孔洞、去除小噪点，得到更完整的珍珠轮廓候选区域。

最后，调用 `cv2.findContours` 提取轮廓，结合面积过滤（保留面积 > 500 的轮廓）初步筛选珍珠轮廓，为精准定位做基础。

1.2 轮廓去重与位置排序

对筛选出的轮廓，先按面积降序排序（`sorted` 结合 `cv2.contourArea`），避免小噪点轮廓干扰。通过**空间距离去重**（计算轮廓质心间距，阈值设为 100 像素），过滤同一珍珠的重复碎片轮廓，保留最多 8 个有效珍珠轮廓。

为实现“从左到右、从上到下”的视觉逻辑排序，设计**相对位置排序算法**：提取所有轮廓质心的 y 坐标，以中位数为界划分“上行”（y 小于中位数）和“下行”（y 大于等于中位数）；每行内按质心 x 坐标升序排列，最终合并为 P1-P8 的有序序列，完成珍珠位置精准定位。

2、计算珍珠圆度

2.1 轮廓特征计算

对已定位的珍珠轮廓，借助 **OpenCV 矩运算**（`cv2.moments`）计算质心（`cx`, `cy`），通过 `cv2.contourArea` 和 `cv2.arcLength` 分别获取轮廓面积 `area` 与周长 `perimeter`。基于圆度公式

$$roundness = \frac{4\pi \cdot area}{(perimeter)^2}$$

遍历每个珍珠轮廓，计算其圆度值，量化珍珠接近正圆的程度。

2.2 结果输出与验证

将计算得到的珍珠编号（P1~P8）、质心坐标、面积、周长、圆度值，通过格式化打印输出表格，清晰呈现各珍珠的圆度指标。同时，生成**标记图**（`cv2.drawContours` 绘制绿色轮廓、`cv2.circle` 标记红色质心、`cv2.putText` 添加蓝色编号），直观验证轮廓定位与圆度计算的准确性。

三、实验结果

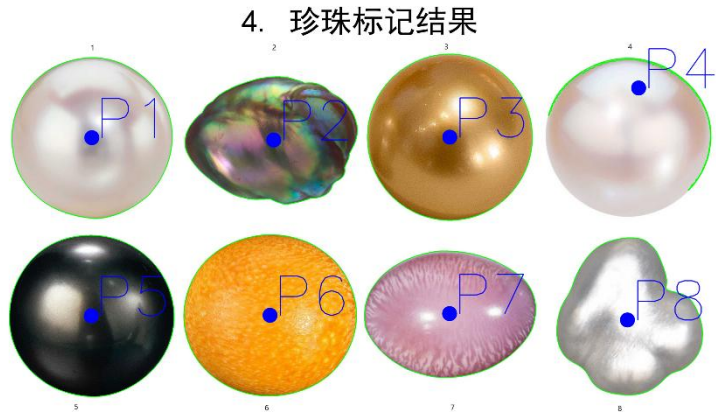
珍珠编号	1 号	2 号	3 号	4 号
中心坐标	(516, 523)	(1489, 535)	(2430, 520)	(3383, 519)
圆度	0.8931	0.8393	0.8938	0.8936

珍珠编号	5 号	6 号	7 号	8 号
中心坐标	(513, 1476)	(1469, 1473)	(2428, 1464)	(3380, 1498)
圆度	0.8990	0.8912	0.8597	0.8021

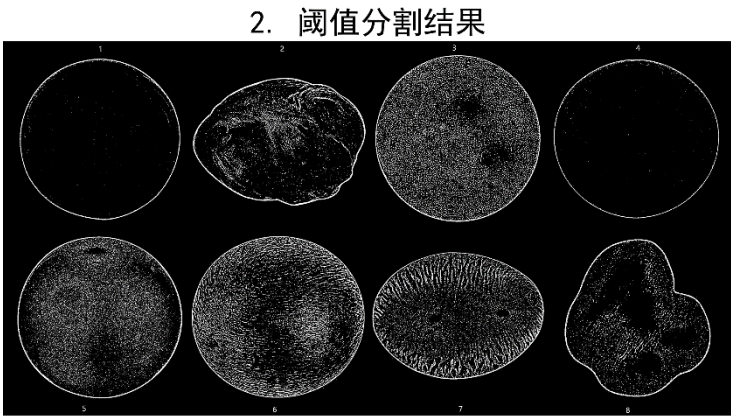
四、总结与体会

1.请记录调试过程中，碰到的问题（至少 3 个），以及相应的解决方法。

（1）珍珠 P4 图形轮廓不完整，如图。



从阈值分割结果图来看，问题根源在于图像对比度低、边缘模糊、噪声干扰。



珍珠与背景的灰度差异小，阈值分割时无法清晰区分边缘。P4 珍珠本身颜色浅，似乎有点和背景“融为一体”，导致轮廓提取不完整；且珍珠表面纹理复杂（光泽、反射），加上图像本身的一些噪声，导致阈值分割后内部出现大量噪点。

所以需要强化珍珠与背景的灰度差异。想到用 **形态学膨胀 + Canny 边缘检测**：

- **膨胀 (cv2.dilate)**：让边缘 “膨胀扩大”，弥补原本模糊的边界；
- **Canny 边缘检测**：精准提取边缘，再通过 bitwise_and 融合回原图，强制让边缘更突出。

于是，加入修改程序段：

```
# 形态学膨胀
kernel_dilate = np.ones((5, 5), np.uint8)
blurred = cv2.dilate(blurred, kernel_dilate, iterations=1)
# Canny 边缘增强
edges = cv2.Canny(blurred, 50, 150)
blurred = cv2.bitwise_and(blurred, blurred, mask=edges)
```

(2) P3、P6 等珍珠内部出现大量白色噪点，导致轮廓提取时出现多个小碎片。

原因分析：珍珠表面光泽反射导致局部亮度不均，且阈值分割算法对复杂纹理处理不足。

于是尝试按面积排序，且增加闭运算环节以填充小孔洞：

```
# 优化轮廓筛选——按面积排序，优先选择最大轮廓
contours = sorted(contours, key=cv2.contourArea, reverse=True)[:8]
#增强形态学处理——增加闭运算迭代次数填充小孔洞
closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)
```

修改后轮廓提取结果中，珍珠内部噪点被有效过滤，只保留完整大轮廓。

(3) 珍珠编号排序不符合预期。

原因分析：原排序算法仅基于中心点坐标，未考虑珍珠的行列布局；图像可能存在倾斜或透视变形，导致绝对坐标排序不准确。

解决方法：行列分组排序——先按 y 坐标中位数分为上下两行，再在每行内按 x 坐标排序。

```
def sort_by_relative_position(features):
    # 计算 y 坐标中位数作为行列分界
    ys = [feat[2] for feat in features]
    y_median = np.median(ys)
    # 分组：上行为 y < 中位数，下行为 y >= 中位数
    upper_row = [f for f in features if f[2] < y_median]
    lower_row = [f for f in features if f[2] >= y_median]
    # 每行内按 x 坐标排序
    upper_row_sorted = sorted(upper_row, key=lambda x: x[1])
    lower_row_sorted = sorted(lower_row, key=lambda x: x[1])
    # 合并顺序：上一行从左到右，下一行从左到右
    return upper_row_sorted + lower_row_sorted
```

效果验证：修改后编号（P1~P8）顺序完全符合从左到右、从上到下的布局。

2.其他——我的一些经验启发

图像预处理对后续分析起到关键作用。起初由于珍珠边缘与背景对比度低，P4 珍珠的轮廓在阈值分割后出现缺失，通过增加形态学膨胀和 Canny 边缘增强操作，有效扩大了珍珠区域并强化了边缘特征，最终使轮廓完整包围珍珠。

轮廓处理需多层筛选，算法设计要贴合视觉逻辑。珍珠表面光泽导致的噪点曾使轮廓提取出现大量碎片，而简单的面积过滤无法完全解决问题，后来通过按面积降序排序并结合空间距离去重，成功过滤了重复轮廓。最初按绝对坐标排序导致编号与视觉顺序不符，改用基于 y 坐标中位数分行、行内按 x 坐标排序的方法后，才实现了从左到右、从上到下的合理编号。

注重细节处理与结果验证。公式应用时需考虑周长为零的边界情况，避免程序异常，同时通过标记图直观校验轮廓提取的准确性，确保圆度值能真实反映珍珠形状。整个实验过程中，问题的解

决离不开对现象的细致观察与原因倒推，从小步迭代修改到可视化结果验证，每一步都需要严谨的逻辑和耐心。

五、带有注释的程序源代码

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设中文字体

def pearl_roundness_measurement(image_path):
    """珍珠圆度测量主函数：从图像中提取珍珠轮廓，计算圆度并按位置排序"""

    # ===== 1. 图像预处理阶段 =====
    # 读取图像并转换为 RGB 格式（OpenCV 默认 BGR）
    img = cv2.imread(image_path)
    if img is None: raise FileNotFoundError("图像读取失败")
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # 转换为灰度图，减少颜色干扰
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # 高斯模糊：消除噪声，平滑图像
    blurred = cv2.GaussianBlur(gray, (5, 5), 0)

    # 形态学膨胀：扩大珍珠区域，解决边缘不完整问题（如 P4）
    kernel_dilate = np.ones((5, 5), np.uint8)
    blurred = cv2.dilate(blurred, kernel_dilate, iterations=1)

    # Canny 边缘增强：突出珍珠轮廓，提高阈值分割精度
    edges = cv2.Canny(blurred, 50, 150)
    blurred = cv2.bitwise_and(blurred, blurred, mask=edges)

    # ===== 2. 阈值分割阶段 =====
    # 自适应阈值分割：根据局部区域亮度自动确定阈值，适应珍珠亮度不均
    thresh = cv2.adaptiveThreshold(
        blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, 11, 2
    )

    # ===== 3. 形态学处理阶段 =====
    # 闭运算：填充小孔洞，连接邻近区域
    kernel = np.ones((3, 3), np.uint8)
    closed = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel, iterations=2)

    # 开运算：去除小噪点和毛刺，平滑珍珠轮廓
```

```

opened = cv2.morphologyEx(closed, cv2.MORPH_OPEN, kernel, iterations=1)

# ===== 4. 轮廓检测与去重阶段 =====
# 提取外部轮廓（只检测最外层边界）
contours, _ = cv2.findContours(opened, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# 过滤小面积轮廓，保留珍珠主体
valid_contours = [c for c in contours if cv2.contourArea(c) > 500]

# 计算每个轮廓的特征（中心、面积、周长、圆度）
features = []
for cnt in valid_contours:
    m = cv2.moments(cnt)
    if m["m00"] == 0: continue # 避免除零错误
    cx, cy = int(m["m10"] / m["m00"]), int(m["m01"] / m["m00"]) # 计算质心
    area = cv2.contourArea(cnt) # 计算面积
    perimeter = cv2.arcLength(cnt, True) # 计算周长
    roundness = 4 * np.pi * area / (perimeter ** 2) if perimeter else 0 # 计算圆度（越接近 1 越圆）
    features.append((cnt, cx, cy, area, perimeter, roundness))

# 按面积降序排序，优先处理大珍珠
sorted_by_area = sorted(features, key=lambda x: x[3], reverse=True)

# 空间去重：过滤距离过近的重复轮廓（可能是同一珍珠的多个碎片）
distance_threshold = 100
unique_features = []
used_indices = set()
for i in range(len(sorted_by_area)):
    if i in used_indices: continue
    unique_features.append(sorted_by_area[i])
    for j in range(i + 1, len(sorted_by_area)):
        if j in used_indices: continue
        dx = sorted_by_area[i][1] - sorted_by_area[j][1]
        dy = sorted_by_area[i][2] - sorted_by_area[j][2]
        if np.sqrt(dx * dx + dy * dy) < distance_threshold:
            used_indices.add(j)

# 保留前 8 个有效轮廓（对应 8 颗珍珠）
filtered_features = unique_features[:8]

# ===== 5. 按相对位置排序阶段 =====
def sort_by_relative_position(features):
    """按行列相对位置排序：先分上下行，再左右排序"""
    centers = [(f[1], f[2]) for f in features]

    # 提取所有 y 坐标，计算中位数作为行列分界

```

```

ys = [cy for (cx, cy) in centers]
y_median = np.median(ys)

# 分组：上行为 y < 中位数，下行为 y >= 中位数
upper_row = []
lower_row = []
for cx, cy in centers:
    if cy < y_median:
        upper_row.append((cx, cy))
    else:
        lower_row.append((cx, cy))

# 每行按 x 坐标从左到右排序
upper_row_sorted = sorted(upper_row, key=lambda x: x[0])
lower_row_sorted = sorted(lower_row, key=lambda x: x[0])

# 合并顺序：上一行从左到右，下一行从左到右
return upper_row_sorted + lower_row_sorted

# 应用排序并匹配特征
sorted_centers = sort_by_relative_position(filtered_features)
sorted_features = []
for cx, cy in sorted_centers:
    for feat in filtered_features:
        if feat[1] == cx and feat[2] == cy:
            sorted_features.append(feat)
            break

# 提取结果
cnts = [f[0] for f in sorted_features]
centers = [(f[1], f[2]) for f in sorted_features]
areas = [f[3] for f in sorted_features]
perimeters = [f[4] for f in sorted_features]
roundness = [f[5] for f in sorted_features]

# ===== 6. 生成标记图阶段 =====
marked_img = img_rgb.copy()
for i, (cnt, (cx, cy)) in enumerate(zip(cnts, centers)):
    # 绘制绿色轮廓，线宽 3 像素
    cv2.drawContours(marked_img, [cnt], -1, (0, 255, 0), 3)

    # 绘制红色实心圆标记中心点，半径 40 像素（便于观察）
    cv2.circle(marked_img, (cx, cy), 40, (0, 0, 255), -1)

    # 添加白色编号，字体大小 10，粗体，位置偏移中心点
    cv2.putText(marked_img, f"P{i + 1}", (cx + 15, cy),

```



```
cv2.FONT_HERSHEY_SIMPLEX, 10, (0, 0, 255), 3, cv2.LINE_AA)
```

```
# ===== 7. 保存图像阶段 =====
```

```
plt.rcParams["figure.dpi"] = 300 # 设置高分辨率（300 DPI）
```

```
# 保存灰度图
```

```
plt.figure(figsize=(8, 6))
```

```
plt.imshow(gray, cmap='gray')
```

```
plt.title('1. 灰度图', fontsize=24)
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# 保存阈值分割结果
```

```
plt.figure(figsize=(8, 6))
```

```
plt.imshow(thresh, cmap='gray')
```

```
plt.title('2. 阈值分割结果', fontsize=24)
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# 保存形态学处理后图像
```

```
plt.figure(figsize=(8, 6))
```

```
plt.imshow(opened, cmap='gray')
```

```
plt.title('3. 形态学处理后', fontsize=24)
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# 保存标记结果图
```

```
plt.figure(figsize=(8, 6))
```

```
plt.imshow(marked_img)
```

```
plt.title('4. 珍珠标记结果', fontsize=24)
```

```
plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# ===== 8. 输出结果表格阶段 =====
```

```
print("珍珠圆度测量结果（按相对位置排序 P1-P8）:")
```

```
print("-" * 70)
```

```
print(f'{{编号:^10}} {{中心坐标:^20}} {{面积:^15}} {{周长:^15}} {{圆度:^15}}')
```

```
print("-" * 70)
```

```
for i, (cx, cy) in enumerate(centers):
```

```
    print(f'P{i + 1:^10} {f'({cx}, {cy})':^20} {areas[i]:^15.2f}
```

```
        {perimeters[i]:^15.2f} {roundness[i]:^15.4f}')
```

```
print("-" * 70)
```

```
return centers, roundness
```

```
if __name__ == "__main__":
    image_path = "pearls1.jpg"
    try:
        centers, _ = pearl_roundness_measurement(image_path)
    except Exception as e:
        print(f'处理错误: {e}')
```

附运行结果:

	编号	中心坐标	面积	周长	圆度
P	1	(516, 523)	585184.00	2869.53	0.8931
P	2	(1489, 535)	480703.50	2682.72	0.8393
P	3	(2430, 520)	623450.00	2960.58	0.8938
P	4	(3383, 519)	593600.50	2889.23	0.8936
P	5	(513, 1476)	603956.50	2905.57	0.8990
P	6	(1469, 1473)	636076.50	2994.88	0.8912
P	7	(2428, 1464)	488177.00	2671.22	0.8597
P	8	(3380, 1498)	467095.50	2705.20	0.8021

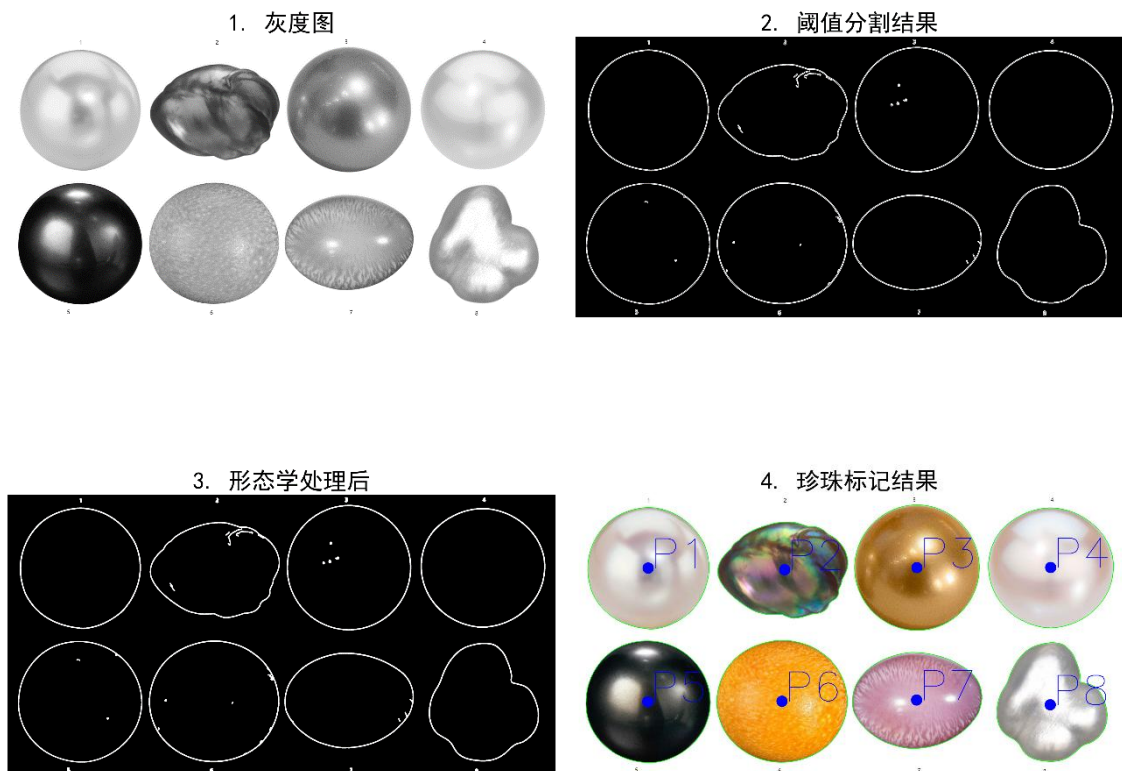


图 灰度图（左上）、阈值分割结果（右上）、形态学处理后（左下）、珍珠标记结果（右下）